

OpenFrame データセットガイド

OpenFrame/Base 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame データセットガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/Base 7.1

ガイドバージョン: v3.1.2

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの`$(INSTALL_PATH)/license/oss_licenses`に記載されている事項を参照してください。

目次

このガイドについて	ix
第1章 データセットの紹介	1
1.1. 概要	1
1.2. データ・アクセス方式	1
1.3. データセット名	4
1.4. ストレージの管理	4
1.4.1. ストレージ・クラス	5
1.4.2. マネジメント・クラス	5
1.4.3. データ・クラス	5
1.5. カタログの登録	6
1.6. データセットの処理	6
1.6.1. データセットの割り当てと割り当て解除	7
1.6.2. データセットのオープンとクローズ	8
1.6.3. オープン・モード	8
1.6.4. アクセス・モード	9
1.6.5. レコード・アクセス	10
第2章 非VSAMデータセット	13
2.1. 概要	13
2.2. レコード形式	13
2.3. DCB構造体	15
2.4. スペースの割り当て	16
2.5. 順次データセット	18
2.6. 区分データセット	19
2.7. 世代データ・グループ	22
2.7.1. 絶対世代番号	23
2.7.2. 相対世代番号	24
2.7.3. マルチステップ・ジョブにおける注意事項	24
2.8. テープ・ボリュームの使用	25
2.8.1. ファイル・シーケンス番号	25
2.8.2. データセットの削除	25
2.8.3. 注意事項	26
第3章 VSAMデータセット	27
3.1. 概要	27
3.2. VSAMデータ形式	27
3.3. VSAMデータセットの種類	29
3.3.1. KSDS（キー順データセット）	30
3.3.2. ESDS（入力順データセット）	30
3.3.3. RRDS（相対レコード・データセット）	31
3.3.4. VRDS（可変長データセット）	31

3.4.	VSAMデータセット名	31
3.5.	代替索引とスフィア	33
3.5.1.	代替索引	33
3.5.2.	スフィア	34
3.6.	VSAMデータセットの管理	34
第4章	統合カタログ	41
4.1.	概要	41
4.2.	カタログ	41
4.2.1.	カタログの構造	41
4.2.2.	カタログの検索順	41
4.2.3.	マスター・カタログ	42
4.2.4.	システム・カタログ	43
4.2.5.	カタログ・エントリ	43
4.2.6.	カタログの情報	44
4.3.	カタログの管理	45
4.4.	関連プログラム	47
付録 A.	カタログ管理API	49
A.1.	概要	49
A.2.	初期化と終了	50
A.2.1.	amsu_initialize()	50
A.2.2.	amsu_finalize()	50
A.3.	ボリューム情報	51
A.3.1.	amsu_volume_list()	51
A.3.2.	amsu_default_volume()	51
A.3.3.	amsu_volume_path()	52
A.4.	カタログ検索	52
A.4.1.	amsu_use_catalogs()	52
A.4.2.	amsu_search_entries()	52
A.4.3.	amsu_master_catalog()	54
A.4.4.	amsu_candidate_catalog()	54
A.5.	ファイル・パス検索	55
A.5.1.	amsu_filepath2()	55
A.5.2.	amsu_filename2()	56
A.6.	エントリ管理	56
A.6.1.	amsu_delete()	56
A.6.2.	amsu_info()	57
A.6.3.	amsu_assoc()	57
付録 B.	データセットI/O API	59
B.1.	概要	59
B.2.	ファイルのオープンとクローズ	60
B.2.1.	tcfh_open()	60
B.2.2.	tcfh_close()	62

B.3.	レコード・アクセス	63
B.3.1.	tcfh_start()	63
B.3.2.	tcfh_read()	64
B.3.3.	tcfh_write()	65
B.3.4.	tcfh_rewrite()	66
B.3.5.	tcfh_delete()	67
B.4.	PDSメンバーの処理	67
B.4.1.	tcfh_bldl()	67
B.4.2.	tcfh_bldl_info()	68
B.4.3.	tcfh_find_by_bldl()	69
B.4.4.	tcfh_find_by_name()	70
B.4.5.	tcfh_stow()	70
B.5.	ファイル・ステータス・コード	71
付録 C.	DCBハンドリングAPI	73
C.1.	概要	73
C.2.	割り振りハンドル	74
C.2.1.	tdcb_get_handle()	74
C.2.2.	tdcb_discard_handle()	75
C.2.3.	tdcb_print_info()	75
C.3.	DCB情報	76
C.3.1.	tdcb_get_type()	76
C.3.2.	tdcb_maxlrecl()	77
C.3.3.	tdcb_nvsm_info()	77
C.4.	属性の取得	79
C.4.1.	tdcb_get_dsname()	79
C.4.2.	tdcb_get_volser()	80
C.4.3.	tdcb_get_flags()	80
C.4.4.	tdcb_get_blksize()	82
C.4.5.	tdcb_get_key_info()	82
C.4.6.	tdcb_get_vsamtype()	83
C.5.	属性の設定	84
C.5.1.	tdcb_set_lrecl()	84
C.5.2.	tdcb_set_recfm()	85
C.5.3.	tdcb_set_blksize()	85
C.5.4.	tdcb_set_keylen()	86
C.6.	動的割り当て	87
C.6.1.	tdcb_allocate()	87
C.6.2.	tdcb_concatenate()	88
C.6.3.	tdcb_unallocate()	89
付録 D.	FCDハンドリングAPI	91
D.1.	概要	91
D.2.	ファイル・オープン・セッション	93

D.2.1.	tfcd_open()	93
D.2.2.	tfcd_close()	94
D.3.	レコード・アクセス	94
D.3.1.	tfcd_start()	94
D.3.2.	tfcd_read()	95
D.3.3.	tfcd_write()	96
D.3.4.	tfcd_writea()	97
D.3.5.	tfcd_delete()	98
D.4.	特殊な操作	98
D.4.1.	tfcd_is_file_open()	98
D.4.2.	tfcd_get_status()	99
D.4.3.	tfcd_get_file_name()	99
D.4.4.	tfcd_get_rec_len()	100
D.4.5.	tfcd_set_file_name()	100
D.4.6.	tfcd_set_relative_key()	101
D.4.7.	tfcd_set_key_id()	101
D.4.8.	tfcd_set_rec_len()	102
付録 E.	OpenFrame VSAMのパフォーマンス最適化	103
E.1.	概要	103
E.2.	その他のパフォーマンス最適化方法	104
索引		107

図目次

[図 2.1]	固定長レコード	14
[図 2.2]	可変長レコード	14
[図 2.3]	区分データセットの構造	20
[図 2.4]	世代データ・グループの概念	22
[図 3.1]	制御インターバル	28
[図 3.2]	代替索引	33

このガイドについて

対象読者

本書は、リホスト・ソリューションであるTmax OpenFrame[®](以下、OpenFrame)/Baseシステムを使用するユーザーを対象としています。

前提知識

本書を理解するには、OpenFrame/Baseについての知識が必要です。本書を読む前に、『OpenFrame Baseガイド』をお読みください。

制限事項

本書では、データセットで現在サポートしていない機能、オプション、パラメータについて説明していますが、それらの詳細については省略しています。

本書で説明しているIDCAMSに対応する日立VOS3システムのVSAMユーティリティはJSCVSUTであり、XSPおよびMSPシステムのVSAMユーティリティはKQCAMSです。

本書の構成

本書は、計4章と付録で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: データセットの紹介

データセットのアクセス方式、データセット名、SMSクラス、カタログの登録と管理およびデータセットの処理など、データセットの全般について説明します。

- 第2章: 非VSAMデータセット

レコード形式、DCB構造体、順次データセット、区分データセット、世代データ・グループなど、非VSAMデータセットの特徴と種類について説明します。

- 第3章: VSAMデータセット

VSAMデータの形式、VSAMデータセットのタイプ、代替索引とスフィア、およびVSAMデータセットの管理方法について説明します。

- 第4章: 統合カタログ

カタログの構造、検索順、マスター・カタログなど、カタログについての全般的な事項やカタログの構造と管理方法について説明します。最後に、カタログ関連のツール・プログラムについて説明します。

- 付録A: カタログ管理API

カタログ管理APIを使用して、カタログ項目とデータセットを管理する方法について説明します。

- 付録B: データセットI/O API

データセットI/O APIを使用して、Cで作成されたアプリケーション・プログラムで、データセットのレコードを読み書きする方法について説明します。

- 付録C: DCBハンドリングAPI

DCBハンドリングAPIを使用して、JCLから割り当てられたDDの属性情報を照会したり、一部の属性を変更したりする方法について説明します。

- 付録D: FCDハンドリングAPI

FCDハンドリングAPIを使用して、メインCOBOLアプリケーション・プログラムでオープンされたデータセットをCで作成されたサブプログラムで読み書きする方法について説明します。

- 付録E: OpenFrame VSAMのパフォーマンス最適化

OpenFrame VSAMのパフォーマンスの最適化方法について説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名、ディレクトリ
<Ctrl>+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上 (32bit, 64bit)
	Linux x86 2.6以上 (32bit, 64bit)
	Linux ia64 2.6以上 (32bit, 64bit)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 (Fixset07)
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、ティーマックスソフトの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame スタートガイド	OpenFrameソリューションの概念的な理解を助けるために作成されています。
OpenFrame Baseインストールガイド	OpenFrame/Base製品のインストールと環境設定について説明しています。
OpenFrame Baseガイド	OpenFrame/Baseシステムの構成要素と全般的な機能について説明しています。
OpenFrame ユーティリティ リファレンスガイド	OpenFrameエンジンと一緒に提供される多様なユーティリティ・プログラムについて説明しています。
OpenFrame ツールリファレンスガイド	OpenFrameシステムの運用に使用する各種ツール・プログラムについて説明しています。
OpenFrame マイグレーションガイド	メインフレーム環境のリソースをOpenFrame環境に移行する際に必要な情報と移行手順および注意事項について説明しています。
OpenFrame エラーメッセージ リファレンスガイド	製品の使用中に発生し得るエラーのタイプ、およびその解決方法について説明しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

参考文献

製品	ガイド
IBMメインフレーム	z/OS DFSMS: Using Data Sets
	z/OS DFSMS: Managing Catalogs
	z/OS DFSMS: Access Method Services for Catalogs
日立メインフレーム	VOS3データ管理の解説書
	VOS3データ型
	VOS3 VSAMの解説書
富士通メインフレーム	OS IV / MSPデータ管理の解説書
	OS IV / MSPデータセットユーティリティ使用手引書
	OS IV / MSP VSAMの解説書

第1章 データセットの紹介

本章では、データセットのアクセス方式、データセット名、SMSクラス、カタログの登録と管理、およびデータセットの処理など、データセットの全般について説明します。

1.1. 概要

データセットとは、論理的に関連付けられているデータ・レコードの集合であり、アプリケーションで使用されるデータの基本単位です。データセットとUNIXファイルの根本的な違いは、データセットを使用する場合、レコード単位のI/Oをサポートするアクセス方式が提供されるという点です。

データセットは、ハードディスクや磁気テープなどの2次記憶装置に格納されます。記憶装置の基本単位はボリュームです。ハードディスクにはあらゆる種類のデータセットが格納できますが、磁気テープには順次データセット(SDS)のみを格納できます。

OpenFrameでは、磁気テープ・デバイスはサポートしていません。その代わりに、ハードディスクの特定領域を磁気テープのボリュームにマッピングする方法を提供します。オプションに応じて磁気テープのボリュームに格納されているデータセットを圧縮して保管できます。

1.2. データ・アクセス方式

データをレコード単位で読み取り、書き込む技術です。それぞれのアクセス方式は、固有のデータセット構造でデータを格納し、データセットを処理するためのインターフェースとユーティリティ・プログラムを持っています。各データセットの処理に使われるアクセス方式は、そのデータセットの構造によって識別されます。たとえば、順次データセットを処理するには、BSAMまたはQSAMを使用する必要があります。

参考

OpenFrameでは、メインフレームのEXCP(EXecute Channel Program)を用いた物理I/O方式は提供していません。一般的なリホスト対象のアプリケーションが、メインフレームの低レベル・アクセス方式のマクロや関数を直接使用することは稀なためです。OpenFrameが目指しているのは、メインフレームのマクロや関数に1対1で対応および互換させることなく、論理的・外部的なレベルの機能や概念の違いに基づいてリホストした場合の高コスト問題を解消するために、低コストで効率的なソリューションを提供することです。

以下は、データセットのアクセス方式の種類と、それについての説明です。

- BDAM (Basic Direct Access Method、基本直接アクセス方式)

プログラムの要求に応じてデータ・レコードを格納し、0から始まるデータ・レコードの相対アドレスに直接アクセスします。BDAMデータセットでは1ブロック・サイズのデータを1つのレコードとして認識します。

BDAMアクセス方式は、最も基本的なアクセス方式です。使用上の制約が多いため、最近ではアプリケーション・プログラムで使用されることは多くありません。

- BSAM (Basic Sequential Access Method、基本順次アクセス方式)

データ・レコードを入力順に格納する方式で構成されたデータセットを順次データセットといいます。プログラムは様々なレコードを1つのブロックとして構成して格納し、BSAMはブロック単位でデータを読み書きできます。

OpenFrameでは、順次データセットを処理するために、データ・ブロックのバッファ管理をユーザーに任せるBSAMアクセス方式ではなく、ユーザーの代わりにバッファを管理するQSAMアクセス方式を提供しています。

- QSAM (Queued Sequential Access Method、待機順次アクセス方式)

BSAMと同様に、入力順にレコードを格納します。QSAMはI/O性能を向上するためにバッファリング手法を使用し、ユーザーの代わりにバッファを管理します。つまり、GETインターフェースで入力バッファから次の順次レコードを読み取り、PUTインターフェースで次の順次レコードを出力バッファに記録します。ディスクに存在するデータを入力バッファに読み込むか、あるいは出力バッファにあるデータをディスクに格納するのはアクセス・メソッドが行います。

- ISAM (Indexed Sequential Access Method、索引付き順次アクセス方式)

ISAMで処理されるデータセットは、索引順次データセットと呼ばれます。ISAMはキー・フィールドの順にレコードを格納し、キー・フィールドでデータ・レコードを照会します。

OpenFrameではISAMデータセットを、内部的にVSAMのKSDSにマッピングしてサポートしていますが、データセット管理の統一性を保つために、VSAMのKSDSに変換して使用することをお勧めします。

- BPAM (Basic Partitioned Access Method、基本区分アクセス方式)

データを区分データセット(PDS: Partitioned Data Set)のメンバーとして格納します。区分データセットの各メンバーは1つの順次データセットとみなされます。

区分データセットは、データセット内部に、それぞれのメンバー名でデータ・アドレスを検索できるディレクトリ情報が含まれています。OpenFrameでの区分データセットは、UNIXファイル・システムのディレクトリで構成されます。

参考

日立メインフレームにはLIMEというアクセス方式が存在しますが、OpenFrameではLIMEには対応していません。LIMEは非VSAMのPDSと同様に、1つのデータセットに複数のメンバーを保存します。一方、実装の面では、VSAMをベースにしてPDSの機能を実装しており、メンバー単位のロック処理が可能であることが特徴です。

- VSAM (Virtual Storage Access Method、仮想記憶アクセス方式)

VSAMは、ディスクに格納された固定長レコードや可変長レコードの直接処理、または順次処理に使用します。VSAMによって構成されたデータは、迅速なアクセスのために必ずカタログに登録する必要があります。

VSAMデータセットは、以下の4つのタイプに分けられます。

種類	説明
ESDS (Entry-Sequenced Data Set、入力順データセット)	入力順にレコードを格納し、新規レコードはデータセットの一番後ろに追加されます。レコードに対する順次処理が可能であり、相対バイト・アドレス(RBA: Relative Byte Address)を使用して直接処理することもできます。
KSDS (Key-Sequenced Data Set、キー順データセット)	レコードはキー・フィールドの昇順に格納され、キー・フィールドでレコードの直接処理を行うことができます。TSAMでは、相対アドレスでKSDSの直接処理ができません。
RRDS (Relative Record Data Set、相対レコードデータセット)	レコード番号(RRN: Relative Record Number)の順にレコードを格納します。このレコード番号でのみレコードを直接処理できます。RRDSには固定長RRDSと可変長RRDSの2種類があります。
LDS (Linear Data Set、線形データセット)	LDSは、データをレコード単位でなく、ブロック単位で格納します。LDSは他のVSAMデータセットと異なり、データセット内にいかなる制御データも持っていません。またTSAMでは、LDSをサポートしていません。

参考

1. OpenFrameではVSAMの対応製品としてTSAMを提供しています。TSAMについての詳細は、[「第3章 VSAMデータセット」](#)を参照してください。
 2. 日立メインフレームではLDSタイプをFDS(Flat Data Set)タイプに区分します。
 3. VSAMデータセットの種類についての詳細は、[「3.3. VSAMデータセットの種類」](#)を参照してください。
-

新規データセットのアクセス方式を選択するには、順次処理をするか直接処理をするか、またどのような処理が行いたいかなどを考慮する必要があります。たとえば、RRDSは直接処理のみを行うアプリケーションや直接処理または順次処理をするアプリケーションに適しています。BSAM(順次データセット)とVSAM ESDSは、順次処理をするアプリケーションに適しています。

1.3. データセット名

ユーザーは新規データセットを作成する際には、一意の名前を付ける必要があります。（一般的にデータセットの名前は、JCLでdsnameで指定されます。）

データセット名は、1つの名前セグメント、または関連付けられた複数の名前セグメントで構成されます。それぞれの名前セグメントは修飾子でレベルを示します。

たとえば、TPLAB.IGKANG.TEST3というデータセット名は、3つの名前セグメントで構成されています。最初のセグメントであるTPLABは「高位修飾子」、最後のセグメントのTEST3は「低位修飾子」と呼ばれます。

データセット名は以下のような規則で作成します。

- 1つの名前セグメントは最大8文字で構成されます。
- セグメントの最初の文字は英字(A-Z)または記号(#、@、\$)のいずれかで始まります。
- セグメントの残りの7文字は、英字(A-Z)、数字(0-9)、記号(#、@、\$)、またはハイフン(-)から選択します。
- 名前セグメントを区切るためにピリオド(.)を使用します。
- データセット名の長さは、名前セグメントとピリオドを含めて44文字に制限されます。

参考

1. IBMメインフレームのTSOや日立および富士通メインフレームのTSS環境では、ユーザーIDで始まるデータセット名が使用できます。該当する環境で、アポストロフィー(')で囲まれていない永久データセット名の先頭にユーザーIDを修飾子として追加し、システムにデータセット名として渡します。
 2. OpenFrame/Batch XSP TJESで全体データセット名は26文字に制限しています。したがって、OpenFrame/Batch XSP TJESを使用する場合、全体データセット名は26文字で作成することを推奨します。
 3. VSAMデータセットの名前は、クラスター名のみ指定し、コンポーネント名を指定しない場合は42文字に制限します。VSAMデータセットの名前については「[3.4. VSAMデータセット名](#)」を参照してください。
-

1.4. ストレージの管理

ジョブを実行するユーザーが、新規データセットを作成するたびに、データセットを格納するユニットやボリューム・シリアルをJCLに直接記述するのは非効率的です。また、ジョブを実行する各ユーザーにストレージ管理を任せると、ストレージ管理への体系がないため、時間が経つにつれ管理が難しくなります。

本節では、OpenFrameシステムのSMSクラス（ストレージ・クラス、マネジメント・クラス、データ・クラス）を使用して、効果的にストレージを管理する方法について説明します。

1.4.1. ストレージ・クラス

ストレージ管理者は、現行システムで使用可能なすべてのストレージに対し、各ストレージ・デバイスの性能または容量を基準にして用途を区別した後、そのストレージ・デバイスを環境設定のsmサブジェクトのストレージ・クラスとして登録します。

システムにストレージ・クラスを登録した後は、JCLのDD文にUNITまたはVOLパラメータを使って直接ボリューム・シリアル番号を指定するのではなく、JCLのDD文にストレージ・クラスを指定することができます。ストレージ・クラスが指定されたデータセットは、そのストレージ・クラスに分類されたストレージ・ボリューム・シリアルのいずれかにデータ領域が割り当てられます。

さらに、データセット名からデータセットの属性が推測できる場合は、JCLのDD文に直接ストレージ・クラスを指定しなくても、自動的にストレージ・クラスが指定されるように設定できます。

ストレージ管理者がACS(Automatic Class Selection)規則を登録すると、そのACSの規則に適したデータセットは、ストレージ管理者が設定したストレージ・クラスに指定されます。また、そのストレージ・クラスに分類されたボリューム・シリアルのいずれかにデータセットのデータ領域が割り当てられます。

参考

1. smsとacsの設定についての詳細は、『OpenFrame 環境設定ガイド』を参照してください。
 2. ストレージ・クラスと似た機能として、日立メインフレームにはストレージ・プール(Storage Pool)という機能があります。これは、同じタイプの装置を複数まとめて1つの論理的な記憶装置として使用することができます。JCLのDD文のUNITパラメータでストレージ・プールの名前を指定して使用します。OpenFrameでは、ストレージ・プールを使用したストレージの割り当て機能はサポートしていません。
-

1.4.2. マネジメント・クラス

ストレージの管理には、新規データセットに対するデータ領域の割り当てや、期限切れデータセットの削除などの操作が含まれます。

ストレージ管理者は、データセットの保管期間に応じてデータセットのクラスを分類した後、マネジメント・クラスとして登録します。登録されたマネジメント・クラスは、前述したストレージ・クラスと同様、JCLのDD文に直接記述して使用するか、またはACS規則を登録して自動的にデータセットにマネジメント・クラスが指定されるように設定できます。

マネジメント・クラスが指定されたデータセットが期限切れになると、OpenFrameのデータセット管理サーバーであるofrdmsvrによって自動的に削除されます。

1.4.3. データ・クラス

データ・クラスは、使用頻度の高いデータセットの属性を基準にクラスを登録します。データ・クラスを使用すると、ユーザーはより簡単にJCLを作成することができます。

データ・クラスに定義されたデータセットの属性は、前述のストレージ・クラスやマネジメント・クラスとは異なり、JCLのパラメータを使って変更できます。

以下は、JCLを使用してデータセットを新規作成する例です。この例では、ACS規則を使用せずに、直接JCLにSMSクラスを指定してデータセットを作成しています。（ストレージ・クラスを直接指定したため、VOLパラメータとUNITパラメータは無効です）

```
//MYJOB    JOB
//STEP1    EXEC PGM=IEFBR14
//MKDS1    DD    DSNAME=IGKANG.TEST.DS0001,DISP=(NEW,CATLG),
//          DATACLAS=TEST,MGMTCLAS=NEVER,
//          STORCLAS=SECURE,VOL=SER=XXXXXX,UNIT=SYSDA
```

1.5. カタログの登録

データセットの物理的なストレージを管理するために、OpenFrameシステムはVTOC(Volume Table of Contents)とカタログを使用します。

VTOCは、1つのボリューム内にあるデータセットの一覧を管理し、データセットに関する基本情報や物理的なディスク・アドレス、データセットのサイズを管理します。カタログは、データセットの多様な属性と、データセットが存在するボリュームに関する情報を記述します。

すべてのデータセットは、以下の方法でカタログに登録されます。

- JCL(Job Control Language、ジョブ制御言語)のDISPパラメータ
- AMS(Access Method Service、アクセス方式サービス)のALLOCATE、またはDEFINEコマンド
- TSO(Time Sharing Option、タイム・シェアリング・オプション)のALLOCATEコマンド

カタログに登録されていない既存のデータセットは、IDCAMS(JSCVSUT/KQCAMS)ユーティリティのDEFINE RECATALOGコマンドを使用してカタログに登録できます。

参考

1. カタログについての詳細は、[「第4章 統合カタログ」](#)を参照してください。
 2. IDCAMSに該当する日立VOS3 VSAMユーティリティはJSCVSUTであり、XSPおよびMSPシステムのVSAMユーティリティはKQCAMSです。IDCAMS(JSCVSUT/KQCAMS)ユーティリティについての詳しい内容は、『OpenFrame ユーティリティリファレンスガイド』を参照してください。
-

1.6. データセットの処理

OpenFrame環境ですべてのプログラムは以下の手順でデータセットを処理します。

1. データセットを割り当てて、プログラムとデータセットを関連付けます。(Allocate)

データセットを処理するには、まずデータセットを割り当てた後、それぞれのアクセス方式が提供するインターフェースを使用してデータにアクセスします。

データセットの割り当てとは、次の2つの作業を意味します。

- 新規データセットのためのディスク領域の割り当て

以下の方法でデータセットを割り当てることができます。

区分	説明
AMS(Access Method Service)	IDCAMS(JSCVSUT/KQCAMS)のALLOCATEコマンドを使用してデータセットを割り当てます。
JCL(Job Control Language)	すべてのデータセットはJCLを使用して直接定義することができます。

- ジョブ・ステップと使用するデータセットを論理的に関連付けること

2. DDNAMEを使用してデータセットを識別し、データセットを開きます。(Open)
3. アクセス方式を使用してデータを読み書きします。(Read/Write)
4. 使用したデータセットを閉じます。(Close)
5. 割り当てられたデータセットを解除します。(Unallocate)

参考

非VSAMデータセットに対する処理手順も同じです。

1.6.1. データセットの割り当てと割り当て解除

ユーザーが既存のデータセットにアクセスする際、システムは物理的なストレージ・デバイスのアドレスを検索するため、JCLでユーザーが提供したデータセットの情報とカタログに登録されているデータセット情報を利用します。また、ユーザーが新規データセットを作成する際、システムはユーザーが提供したデータセットの情報に基づき、ディスクにデータセットのストレージを確保します。このように、既存のデータセットのストレージ・アドレスを検索する作業、または新規データセットのストレージを確保する作業などを**割り当て(Allocate)**といいます。

- データセットを新規作成するために割り当てる場合には、新規データセットが格納されるボリュームとデバイス情報、データセットのサイズ情報が必要です。
- 既存のデータセットを使用するために割り当てる場合には、ボリュームとデバイス情報が必要です。データセットがカタログに登録されている場合は、データセット名を指定するだけで十分です。

システムはデータセットを割り当てるとき、DCB(Data Control Block)と呼ばれる構造体を用意します。DCB構造体は、1つのデータセットを処理するために必要な情報とリアルタイムでI/Oレポートを行うためのバッファで構成されています。

データセットを処理する前に割り当てが必要であり、データセットの処理後には**割り当てを解除 (Unallocate)**する必要があります。JCLで記述されるデータセットのディスポジションは、システムがデータセットを解除する際、どのように処理すべきかを指定します。たとえば、一時データセットの処理において、作業が終わるまでデータセットを維持すべきか、あるいはすぐに削除すべきかを決定し、永久データセットの処理においては、マスター・カタログに登録されるべきか、もしくはユーザー・カタログに登録されるべきかを決定します。

1.6.2. データセットのオープンとクローズ

アプリケーションは特定のデータセットにアクセスする前に、オープン・インターフェースを使用してデータセットを開く必要があります。オープン・インターフェースは、DCBを介してアプリケーションとアクセサメソッドおよびデータセットを論理的に関連付けます。

オープン・インターフェースは、DCB構造体を完成し、データセットを初期化し、データセットの処理に必要なシステム内部バッファを用意します。

DCB構造体は、次の3つの情報により完成されます。

- カタログ情報
- JCL DD文
- アプリケーション

クローズ・インターフェースは、アプリケーションとデータセットの接続を切断します。

データセットを閉じる際、システムは以下のような作業を行います。

- 物理的に記録されていないデータをディスクに書き込みます。
- データセットのサイズ情報などのカタログ情報を変更します。
- データセットの処理に使用されたDCB構造体を解除します。

1.6.3. オープン・モード

データセットのオープン・モードを指定します。アプリケーションのデータセットに対する読み書きを設定するためのモードです。

データセットは、次の4つのオープン・モードのいずれかに指定されます。

- 入力モード(Input Mode)

入力モードを指定すると、データセットの読み取りのみ可能になります。レコードを記録、変更、削除などの操作にはエラーが発生します。存在しないデータセットを開く場合、OPTIONALオプションを指定していなければエラーが発生します。

- 出力モード(Output Mode)

出力モードを指定すると、アプリケーションがレコードを書き込みできる新規データセットを作成します。データセットでレコードを読み取る操作にはエラーが発生します。また、書き込みの目的で既存のデータセットを開くと、既存のすべてのレコードは削除されます。

- 入出力モード(Input/Output Mode)

入出力モードを指定すると、データセットの読み書き操作が可能になります。存在しないデータセットを開く場合、OPTIONALオプションを指定していなければエラーが発生します。

- 拡張モード(Extend Mode)

拡張モードは出力モードと似ていますが、既存のデータセットへの書き込みにのみ使用されます。アプリケーションはファイルの末尾に新規レコードを書き込みます。存在しないデータセットを開く場合、OPTIONALオプションを指定していなければエラーが発生します。

参考

該当するデータセットが存在しない場合、OPTIONALオプションが指定されていれば、データセットを作成します。

1.6.4. アクセス・モード

データセットのアクセス・モードは、データセットに存在するレコードの処理順を意味しており、データセットの属性ではなくアプリケーションでデータセットを使用する方法を示します。

次の3つのアクセス・モードのいずれかを指定します。

- 順次アクセス(Sequential Access)

順次アクセス・モードは、レコードを最初のレコードから最後のレコードまで順次に読み書きする方法です。ESDS、RRDS、KSDSはすべてのレコードに対する順番が存在するため、順次にアクセスすることができます。RRDSの順番は相対レコード番号(RRN)によって決まり、KSDSの順番は主キーまたは補助キーのいずれの英数字のキーで決まります。

- ランダム・アクセス(Random Access)

ランダム・アクセス・モードは、希望する順にレコードを処理します。ESDS、RRDS、KSDSはランダム・アクセス・モードをサポートします。ランダム・アクセス方式でデータセット・レコードを読み取る場合、ユーザーはデータセットの構成方式に応じて、RBA、RRN、キー値などのレコード識別子を指定する必要があります。

- 動的アクセス(Dynamic Access)

動的アクセス・モードは、アプリケーションを実行している間、データセットに対して順次アクセス・モードとランダム・アクセス・モードを両方とも使うことができます。ESDS、RRDS、KSDSは動的アクセス・モードをサポートします。

1.6.5. レコード・アクセス

データセットがオープンされると、TSAMで提供するさまざまなインターフェースを使用して、データセットに存在するレコードを処理することができます。たとえば、READインターフェースを介してデータセットに存在するレコードを読み取り、WRITEインターフェースを使用してデータセットに新規レコードを追加することができます。

次の4つのレコード・アクセス・インターフェースを使用してデータセットのレコードを管理します。

- WRITE（レコードの挿入と追加）

TSAM ESDSでは、新規レコードを間に追加することができません。TSAM KSDSやRRDSの場合、キーを使用して追加されるレコードの位置を指定すると、間にレコードを追加することが可能です。該当のレコード・キーにすでにレコードが存在する場合は、DUPLICATE RECORDエラーコードが返されます。

- READ（レコードの読み取り）

- 順次アクセス

アプリケーションがデータセットに初めて順次アクセスを実行すると、常に最初のレコードから読み取り始めます。以前、データセットにランダム・アクセスを行ったことがある場合は、内部のレコード・ポインターが変更されている可能性があるため、STARTインターフェースを使ってレコード・ポインターを初期化してから、順次アクセス・インターフェースを使用することをお勧めします。

- ランダム・アクセス

TSAM ESDS、KSDS、RRDSはレコード識別子を使用してランダム・アクセスをサポートします。ランダム・アクセス・モードは内部レコード・ポインターの影響を受けません。読み取るレコードを検索するには、TSAMデータセットの種類に応じて、以下の情報を提供する必要があります。

データセット	情報
KSDS	KEY
RRDS	RRN
ESDS	RBA

参考

ESDSに対してもRBAを使用したランダム・アクセスは可能ですが、一般的な業務アプリケーションで使用するには適していません。

- REWRITE（レコードの変更）

REWRITEインターフェースを使用して、レコードの内容を変更することができます。ただし、TSAM KSDSのレコードを変更するとき、主キーの内容は変更できません。

- DELETE（レコードの削除）

DELETEインターフェースを使用して、データセット・レコードを削除することができます。TSAM RRDSでレコードを削除すると、削除されたレコードが使用していた領域を再使用することができます。ただし、TSAM ESDSではレコードを削除することができません。

第2章 非VSAMデータセット

本章では、レコード形式、DCB構造体、順次データセット、区分データセット、世代データセット・グループなど、非VSAMデータセットの特性と種類について説明します。

2.1. 概要

非VSAMデータセットとは、IBMのメインフレームでVSAMデータセットを提供する前に使われていたすべてのデータセットをいいます。したがって、VSAMデータセットでないデータセットは、すべて非VSAMデータセットです。

OpenFrameでは非VSAMデータセットをUNIXのファイル・システムで対応し、各データセットの種類に応じたアクセス方式は、別のOpenFrameインターフェースを提供しています。

2.2. レコード形式

OpenFrameでは、次の4つのレコード形式をサポートしています。

- 固定長(RECFM=F)
- 可変長(RECFM=V)
- 行順次(RECFM=L)
- 不定長(RECFM=U)

行順次レコード形式と不定長レコード形式以外のレコード形式は、RECFMパラメータに「B」を追加することでブロックI/Oが可能になります。データ・ブロックは、ディスク・ボリュームにデータが記録される前に、複数のレコードをまとめて1つのブロックとして構成したものです。

OpenFrameでは、RECFMパラメータに「B」が指定されていなくても、実際にディスクI/Oを実行するとき、内部バッファーを使用してバッファリングを行います。

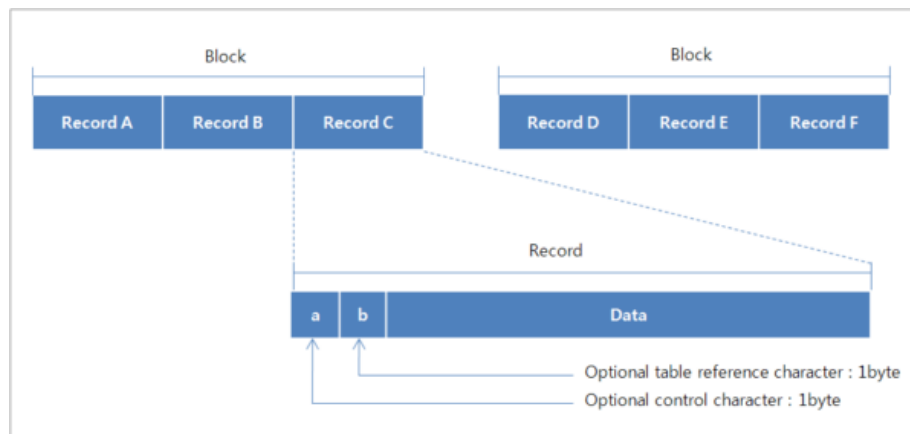
固定長レコード形式

1つのデータセットにあるすべてのレコード・サイズは同じであり、ブロックI/O(RECFM=FB)をサポートします。ブロックI/Oでない場合、1つのレコードは1つのブロックとしてみなされます。

各レコードは、1つのブロック内で別のレコード区切り文字(Record Separator)なしで連続して保存されます。また、レコードを順次に読み取る場合、1つのブロックの中で、一度にレコードのサイズずつ読み取ります。

選択制御文字(Optional Control Character)は、各レコードをプリンターで出力するとき、キャリッジを制御する文字です。また、選択テーブル参照文字(Optional Table Reference Character)は、レコードの出力時にフォントを選択するためのコードです。

[図 2.1] 固定長レコード

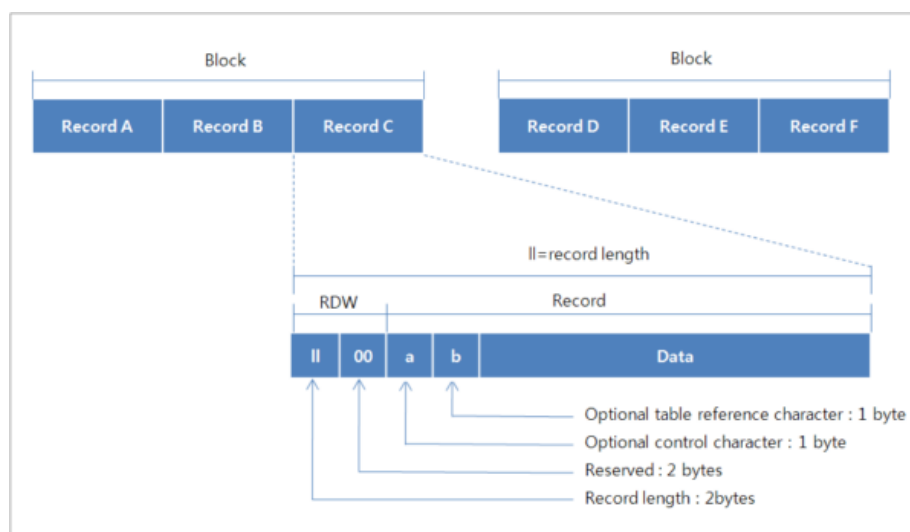


可変長レコード形式

可変長データセットに含まれている各レコードは、それぞれ異なる長さにすることができます。可変長レコード形式の場合、ブロックI/O(RECFM=VB)とスパン形式(RECFM=VS)と一緒に使用することができます。

スパン形式とは、論理レコードの長さが指定されたブロック・サイズより大きい場合、それをセグメント単位で分けてディスクに格納する形式です。OpenFrameでは、論理レコードをセグメント単位で分けて格納するのではなく、メインフレームとは異なる方式でブロックI/Oを実装しています。したがって、スパン形式が指定された状態で論理レコードの長さがブロック・サイズより大きい場合でも、読み書き動作が問題なく実行されます。

[図 2.2] 可変長レコード



可変長のブロックは、BDW(Block Descriptor Word)と1つ以上の論理レコードから構成されています。BDWは、ブロックを付加説明する4バイト・サイズのフィールドです。OpenFrameではBDWは使用されません。

可変長レコードは、RDW(Record Descriptor Word)とデータから構成されます。RDWは、レコードを付加説明する4バイトのフィールドです。RDWの最初の2バイトは、RDWを含む論理レコードの長さを格納します。そして、3、4番目のバイトは、いずれも0に設定されています。

行順次レコード形式

行順次レコード形式は、UNIXのテキスト・ファイルと同じタイプのレコード形式です。ブロック単位のI/Oはサポートせず、データセット内の各レコードは、改行(LF)文字で区切られます。

OpenFrame/Batchのジョブで使用されるインストリーム・データセットやスプール出力データセットは、行順次レコード形式を使用します。

不定長レコード形式

不定長レコード形式は、特に決められた形式なしで入出力を行うためのレコード形式です。不定長レコード形式において、1ブロックは1つのレコードとみなされます。

選択制御文字

JCLのDD文やDCBパラメータには、データセット・レコードの一部として選択制御文字が含まれているかどうかを示すことができます。各レコードの先頭にある1バイトの出力制御文字は、データセットが出力されるときにキャリッジの制御を行います。この1バイトの文字は、レコードの一部としてデータセットに保存されますが、出力される際には表示されません。この制御文字の使用有無は、データ制御ブロック(DCB)のRECFMフィールドに「M」または「A」文字を指定することで決まります。「M」はマシン・コードを、「A」はANSIコードを意味します。

2.3. DCB構造体

OpenFrameでは、アプリケーションによってデータセットがオープンされる前に、処理するすべてのデータセットのデータ制御ブロック(DCB)が用意されている必要があります。データ制御ブロックは、データセットの適切な処理情報を持つ構造体であり、JCLのDD文、またはカタログのデータを抽出して構成されます。

一般的なJCLバッチ・ジョブを実行すると、システム内部でアプリケーションで必要とするすべてのデータセットの制御ブロックをJCLのDD文から作成し、アプリケーションに渡します。ただし、動的割り当てを使用するIDCAMS(JSCVSUT/KQCAMS)のようなユーティリティを使用して、直接データ制御ブロックを作成することもできます。

以下は、データ制御ブロックに指定されるデータセット情報の一部です。

項目	説明
Block Size (BLKSIZE)	入出力のためのデータ・ブロックの最大サイズを指定します。 指定しない場合、デバイスの種類によって32KBまたは64KBをデフォルト・ブロック・サイズとして使用します。OpenFrameでは、ブロック・サイズ・パラメータをサポートしていますが、実際にディスクI/Oを実行するときは、OpenFrame環境設定のdsサブジェクトの DATASET_DEFAULT セクションの NVSM_BUFFUR_SIZE キーに設定されているバッファ・サイズが使用されます。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
Data Set Organization (DSORG)	データセットの構造を指定します。 以下のような種類のデータセット構造を指定できます。 – PS (Physical Sequential) – IS (Indexed Sequential) – PO (Partitioned) – DA (Direct)
Record Format (RECFM)	RECFMは、以下のようなレコードの特徴を持ちます。 – 固定長(F) – 可変長(V) – 行順次(L) – 不定長(U) ブロック単位の入出力をサポートする場合のRECFMはFBとVBが使用され、スパン形式のレコードをサポートする場合のRECFMはVS、VBSが使用されます。
Record Length (LRECL)	LRECLは、データセット・レコードの長さを指定します。可変長レコード形式または不定長レコード形式の場合は、最大レコード長を指定する必要があります。

2.4. スペースの割り当て

非VSAMデータセットのスペース・サイズを割り当てる方法について説明します。

1次スペースと2次スペースの決定

1次スペース(Primary Space)と2次スペース(Secondary Space)のサイズは以下のように決まります。

1. データ・クラスのAVGREC、AVGVAL、PRIMARY、SECONDARYの値を使用する場合

```
Primary Space = (PRIMARY * AVGVAL * AVGREC + 1023)/1024
Secondary Space = (SECONDARY * AVGVAL * AVGREC + 1023)/1024
```

項目	説明
PRIMARY	デフォルト値 : 4096
SECONDARY	デフォルト値 : 0
AVGVAL	CYL(768KB)、TRK(48KB)、またはLRECLに指定した値

2. JCL DD文にLIKEまたはDCB=(dsname)が指定されている場合

モデルとなるデータセットの1次スペースと2次スペースの値を使用します。

3. JCL DD文のSPACEパラメータが指定されている場合

SPACE=(AVGVAL、PRIMARY、SECONDARY)パラメータに対し、以下のように設定します。

```
Primary Space = (PRIMARY * AVGVAL * AVGREC + 1023)/1024
Secondary Space = (SECONDARY * AVGVAL * AVGREC + 1023)/1024
```

項目	説明
PRIMARY	デフォルト値 : 4096
SECONDARY	デフォルト値 : 0
AVGVAL	CYL(768KB)、TRK(48KB)、またはLRECLに指定した値

4. 1から3の方法で1次スペースが決まらなかった場合

```
Primary Space = ボリュームデバイスの最大スペース値 (MB単位で指定)
Secondary Space = 0
```

1次スペースと2次スペースで最大サイズを計算

決定された1次スペースと2次スペースの値を使って最大サイズを計算します。

```
Data Set Size Limit = (Primary Space + Secondary Space * conf_extent_limit) * 1024
```

項目	説明
conf_extent_limit	OpenFrame環境設定のdsサブジェクトのDATASET_DEFAULTセクションのNVSM_EXTENT_LIMITキーのVALUE項目の値であり、2次エクステン트의最大回数を意味します。指定しない場合、デフォルト値は15です。 dsサブジェクトのDATASET_DEFAULTセクションに指定したDATASET_SIZE_LIMITキーのVALUE項目の値がデータセットの最大サイズ

項目	説明
	より小さい場合は、 DATASET_SIZE_LIMIT キーの値を使用し、その逆の場合は、上記で計算したデータセットの最大サイズを使用します。

参考

OpenFrame環境設定の詳細については、『OpenFrame 環境設定ガイド』を参照してください。

2.5. 順次データセット

順次データセットは、OpenFrameシステムで最も一般的なデータセットです。レコードが格納された順にデータ処理が可能なデータセットです。順次データセットは、ハードディスクのボリュームやテープ・デバイス・ボリュームに格納することができます。

順次データセットの作成

QSAMまたはBSAMアクセス方式を使用すると、以下のような手順で順次データセットを作成することができます。

1. DCBパラメータにDSORG=PSと指定します。
2. DD文にデータセットの属性を記述します。
3. データセットを開き、PUTまたはWRITE APIを実行した後、閉じます。

順次データセットの検索

以下は、順次データセットでレコードを読み取る手順です。

1. DCBパラメータにDSORG=PSと指定します。
2. DD文にデータセットの位置を指定します。
3. データセットを開き、GETまたはREAD APIを使用した後、閉じます。

順次連結されたデータセット

OpenFrameシステムでは、2つ以上のデータセットを連結して、1つのデータセットであるかの連続的に検索できる方法を提供します。これを順次連結(Sequential Concatenation)といいます。

順次連結されたデータセットは逆読みができず、VSAMデータセットは順次連結をサポートしません。

以下は、BSAMを使用して複数のデータセットを順次連結する例です。

```
//INPUT DD *  
        ... (instream data set)  
//      DD DSN=IGKANG.MAIN.DATA, DISP=SHR  
//      DD DSN=IGKANG.SUPL.DATA, UNIT=3390, DISP=OLD,  
        BLKSIZE=4096
```

順次データセットの変更

以下は、順次データセットを変更する2つの方法です。

- 既存のレコード内容を変更(Update in place)

既存のレコードの内容を変更するには、まず、レコードの内容を読み取って処理した後、変更されたレコードの内容を元の位置に書き込む必要があります。そのとき、以下のような規則が適用されます。

- 既存のレコードは削除できません。
- データセットはハードディスクのボリュームに存在している必要があります。

- データセットの最後に新規レコードを追加(Extends the data set)

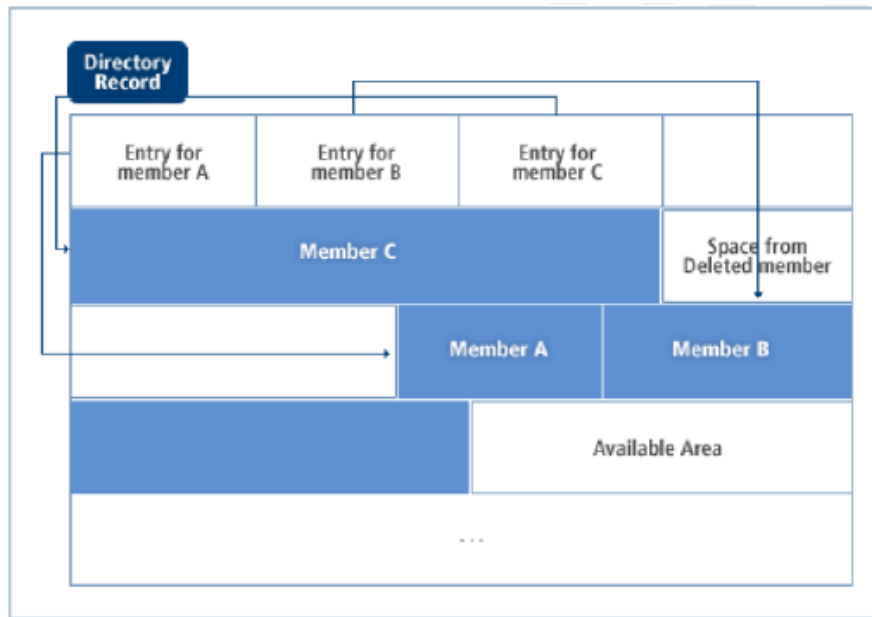
データセットの最後に新規レコードを追加するには、DD文にDISP=MODを指定し、データセットをOUTPUTモードで開きます。その後、PUTまたはWRITE APIを使用することができます。順次データセットでの新規レコードは、常に最後のレコードとして追加されます。レコードとレコードの間に新規レコードを挿入することはできません。

2.6. 区分データセット

区分データセット(PDS)は、複数の順次構造を持つメンバーと、それぞれのメンバーを記述するディレクトリ項目で構成され、ハードディスクのボリュームにのみ格納されます。

各メンバーは、最大8文字の一意的な名前を持ち、メンバーの各レコードは順次に読み書きすることができます。

[図 2.3] 区分データセットの構造



区分データセットのメリットは、全体のデータセットを検索しなくても個別メンバーへのアクセスが可能である点です。たとえば、プログラム・ライブラリの場合、PDSの各メンバーは個別に実行可能なプログラムまたはサブルーチンです。必要に応じて、各メンバーの追加または削除が可能です。

OpenFrameでは、区分データセットをUNIXファイル・システムのディレクトリにマッピングし、メンバーは該当するディレクトリ内にある個別ファイルとして管理します。

区分データセットの作成

BSAM、QSAM、またはBPAMアクセス方式を使用して、区分データセットとPDSのメンバーを作成することができます。

以下は、データセットを作成してメンバーを追加する方法です。

1. DCBパラメータにDSORG=PSと定義します。
2. DD文に、データが新規PDSのメンバーとして格納されることを定義します。つまり、DSNAME=name(membername)、DISP=NEWと定義します。
3. データセットを開き、PUTまたはWRITE APIを実行した後、閉じます。

以下は、区分データセットを作成するJCLの例です。

```
//PDSDD DD ---,DSNAME=MASTFILE(MEMBERK),SPACE=(TRK,(100,5,7)),  
// DISP=(NEW,CATLG),DCB=(DSORG=PS,RECFM=FB,LRECL=80),---
```

DSORG=PSの代わりにDSORG=POを指定し、PUT APIまたはWRITE APIの代わりにSTOW APIを使用すると、BPAMアクセス方式を使用してPDSを作成することができます。

既存のPDSに新規メンバーを追加する方法は前述した手順と同様ですが、ディスポジションだけ「変更」に指定します。(DISP=MOD)

以下は、一度に複数のメンバーを追加する方法です。

1. DCBパラメータにDSORG=POと指定します。
2. DD文でPDS全体のデータセットの位置を指定します。
3. データセットを開き、STOW APIを使用して新規メンバーを追加します。

PDSメンバーの処理

PDSのメンバーは、一般の順次データセットと同じであるため、順次データセットの処理方法と同様にPDSのメンバーの入出力を行うことができます。ただし、ディレクトリ項目の管理やメンバーを検索するために、OpenFrameでは以下のような別のAPIを提供しています。

BLDL APIを使用すると、PDSのディレクトリ項目についての情報をすべて検索してメモリに読み込みます。FIND APIを使用すると、指定されたメンバーの位置にレポート・ポインターを移動させます。一方、STOW APIを使用すると、新しいメンバーの追加、既存メンバーの削除、メンバー名の変更などが可能になります。

上記のAPIを使用するためには、DCBパラメータにDSORG=POを指定する必要があります。

以下は、BPAMアクセス方式の各APIについての説明です。

API	説明
BLDL	ディレクトリ・リストを構成します。 BLDL APIを使用すると、ディレクトリ項目の情報をメモリに読み込みます。この情報を利用して、FIND APIからPDSのメンバーにアクセスできます。
FIND	メンバーの開始アドレス位置を取得します。 FIND APIを使用すると、レコード・ポインターを特定のメンバーの前に移動することができます。以降のI/O処理は、FINDによって指定された位置から実行されます。
STOW	ディレクトリをアップデートします。 1つ以上のメンバーをPDSに追加したい場合、STOW APIを使用してメンバーに該当するディレクトリ項目をPDSディレクトリに直接追加することができます。STOW APIを使用するには、DCBパラメータにDSORG=POを指定する必要があります。

PDSメンバーの検索

PDSで特定のメンバーを検索したい場合は、以下のようにBSAMまたはQSAMを使用します。

1. DCBパラメータにDSORG=PSと指定します。
2. DD文で既存のPDSのメンバーについて記述します。つまり、DSNAME=name(membername)と、DISP=OLD、DISP=SHR、またはDSIP=MODを指定します。
3. データセットを開き、GETまたはREAD APIを使用した後に閉じます。

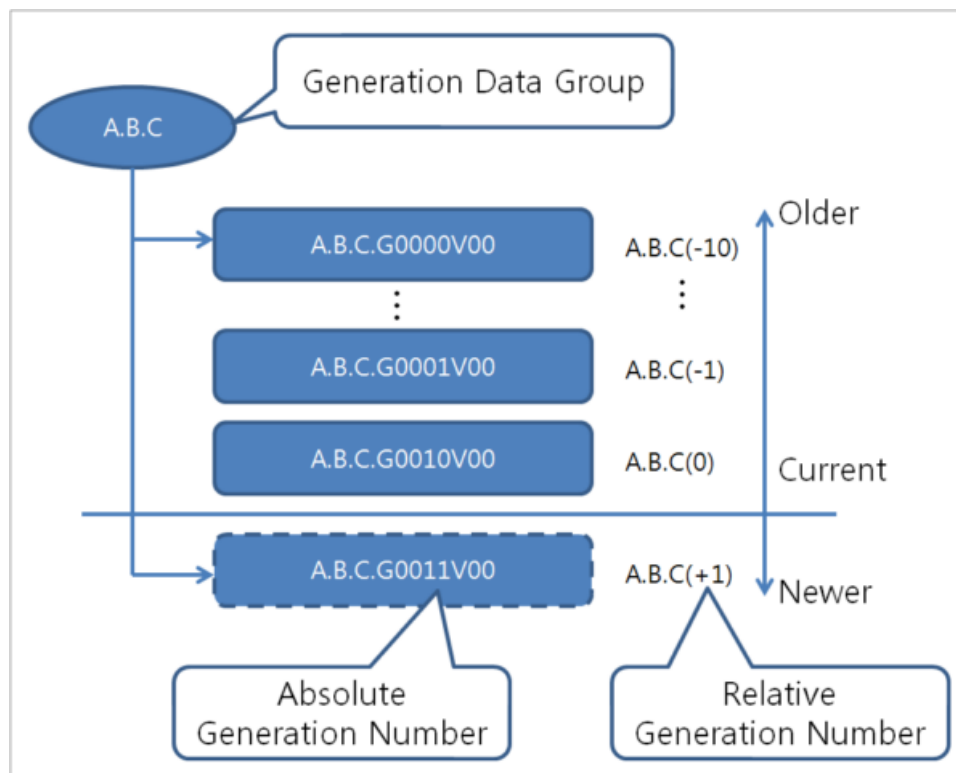
以下は、PDSのメンバーを処理するJCLの例です。

```
//PDSDD DD ---,DSN=MASTFILE(MEMBERK),  
// DISP=SHR,DCB=(DSORG=PS),---
```

2.7. 世代データ・グループ

OpenFrameでは、相互関係のある連続したデータセットをカタログする方法を提供します。これを世代データ・グループ(GDG:Generation Data Group)といいます。GDG内に含まれている各データセットを世代データセット、または世代と呼びます。

[図 2.4] 世代データ・グループの概念



世代データセットは、順次データセットでのみ構成でき、区分データセットやVSAMデータセットは世代データセットにはなれません。

相互関係のあるデータセットを世代データ・グループにまとめると、以下のようなメリットがあります。

- 同じグループにあるデータセットは、同じ名前で参照できます。
- システムはデータセットの世代を順番に管理することができます。
- 古い世代のデータセットは、自動的にシステムから削除されます。

世代データセットは、自身の年齢を意味する絶対名と相対名を持っています。カタログ管理システムは、絶対世代名のみを使用します。年齢が多いデータセットは、小さい絶対数字を持っています。相対名の場合、最新のデータセットは(0)、その次の最新データセットは(-1)のような数字で表示されます。相対名は、新規世代(+1)をカタログ化するためにも使われます。

世代データ・グループ(GDG) BASEは、世代データセットがカタログされる前にカタログに登録されている必要があります。各世代データ・グループは、カタログされているGDG BASE項目によって代表されます。gdgcreateツールを使用して、GDG BASEをカタログに登録することができます。

参考

gdgcreateプログラムについての詳細は、『OpenFrame ツールリファレンスガイド』を参照してください。

2.7.1. 絶対世代番号

絶対世代番号とバージョン番号は、GDGの世代を表すために使用されます。絶対世代番号とバージョン番号は、GxxxxVyyの形式を持ちます。xxxxは4桁の世代番号(0001から9999まで)、yyは2桁のバージョン番号(00から99まで)を示します。

以下は、絶対世代番号とバージョン番号の例です。

例) A.B.C.G0001V00

上記例は、A.B.Cという世代データ・グループの世代番号1、バージョン番号0のデータセットを表しています。

例) A.B.C.G0009V01

上記例は、A.B.Cという世代データ・グループの世代番号9、バージョン番号1のデータセットを表しています。

システムは世代番号を自動的に管理します。維持される世代の最大数は、GDG BASEに定義された限度(Limit)によって決まります。たとえば、限度が10のGDGは、最大10個の項目がGDGによって管理されます。

バージョン番号は、GDGによる世代管理を妨害せずに、一般的なデータセットの処理ができるようにします。たとえば、3つの世代が管理されているGDGで2番目の世代をアップデートしたい場合、

G0002番の世代でV00番のバージョンをG0002番の世代でV01番のバージョンに変えることができます。1つの世代には1つのバージョンのみ、GDGによって管理されます。

世代データセットをカタログに登録する場合は、絶対名と相対名の両方を使用できます。世代が登録された場合は、その世代の世代番号がGDG項目に登録されます。バージョン番号がV00以外の世代データセットに登録するには、絶対世代とバージョン番号を使用する必要があります。

2.7.2. 相対世代番号

世代を指定するとき、絶対世代とバージョン番号を使用する代わりに、相対的な世代番号を使用することができます。相対世代番号を使用するには、A.B.C(-1)、A.B.C(+1)、A.B.C(0)のように、GDG名の後に括弧で囲まれた数字を指定します。

相対世代番号が与えられたら、システムは登録された最新の絶対世代番号を参考にして、与えられた相対世代番号に対応する絶対世代番号を検索します。たとえば、以前のジョブでA.B.C.G0005V00が最後にカタログされた世代データセットであるとした場合、A.B.C(+1)が与えられると、システムはA.B.C.G0006V00の絶対番号を割り当てます。

2.7.3. マルチステップ・ジョブにおける注意事項

マルチステップ・ジョブでデータセットをカタログ化あるいはアンカタログ化する場合は、IEHPROGM(JSGPROGM)や他のユーザー・プログラムを使用せずに、JCLのみを使用する必要があります。IEHPROGM(JSGPROGM)プログラムやユーザー・プログラムの実行中に割り当てられたデータセットは、データセット割り当て管理システムに通知されないため、その後のジョブ・ステップでデータセットの現在状態が失われ、衝突が生じるなどの問題が発生する可能性があるためです。

以前にカタログされた世代データセットの相対番号を使用して参照する場合、相対番号は以下のような意味を持ちます。

- A.B.C(0)は、最後に登録された項目を示します。
- A.B.C(-1)は、最後に登録された項目の1つ前の項目です。

JCLを使用してデータセットをカタログ化すると、実際のジョブはジョブ・ステップの終了時に実行されます。ただし、相対世代番号は実際のジョブが終了するまで保持されます。そのため、以下のような現象が起こります。

- JCLで使用される相対世代番号は、バッチ・ジョブの実行中には常に同じ世代データセットを示します。
- 世代データセットのGDGへのカタログ化が正常に終了したステップを再実行する場合、JCLをサブミットする前に相対世代番号をすべて変更する必要があります。たとえば、A.B.C(+1)はA.B.C(0)に、A.B.C(0)はA.B.C(-1)に、A.B.C(-1)はA.B.C(-2)に変更しなければなりません。

2.8. テープ・ボリ्यूムの使用

テープ・ボリ्यूムには、非VSAMデータセットを保管することができます。VSAMデータセットの保管はサポートしていません。

OpenFrameでテープ・ボリ्यूムを割り当てて使用する場合は、実際に物理テープ・デバイスを使用するのではなく、DASDにボリ्यूムをマッピングして使用し、テープ・ボリ्यूムを仮想で使用するようになります。テープ・ボリ्यूムはvolmgrツールを使って作成できますが、テープ・デバイスを定義する際にデバイス・タイプ番号を3480に指定する必要があります。

```
volmgr define device -dn 0010 -dt 3480 -dg STAPE
```

そして、このテープ・デバイスに属する論理テープ・ボリ्यूムと論理テープに属する物理テープ・ボリ्यूムを定義します。

```
volmgr define volume -v STAPE1 -dn 0010  
volmgr define volume -v ST0001 -lv STAPE1 -t
```

論理テープ・ボリ्यूムには、物理テープ・ボリ्यूムを2つ以上作成することができます。必要に応じてボリ्यूムをスイッチして使用することもできます。

```
volmgr update volume -v STAPE1 -pv ST0002
```

2.8.1. ファイル・シーケンス番号

OpenFrameは、ファイル・シーケンス番号を利用したテープ・ボリ्यूムのアクセスをサポートします。

ファイル・シーケンス番号を利用するためには、OpenFrame環境設定のdsサブジェクト、**DATASET_DEFAULT**セクションの**USE_TAPE_FILESEQ**キーの値をYESに指定します。

```
ofconfig update -n NODE1 -s ds -sec DATASET_DEFAULT -k USE_TAPE_FILESEQ -v YES
```

ファイル・シーケンス番号は、JCLを使ってデータセットにアクセスする場合に利用できます。XSPシステムではシーケンス番号をFILEに記述し、その他のシステムではDD文のLABELパラメータに記述します。シーケンス番号を利用すると、テープ・ボリ्यूムでシーケンス番号に移動してデータセットを探します。データセット名は、シーケンス番号の位置にあるデータセットとの整合性を確認するために、または新規データセットを作成するときに利用されます。シーケンス番号が省略された場合は、データセット名でデータセットの場所を特定します。指定したデータセット名が存在しない場合は、シーケンス番号1番と見なされます。

2.8.2. データセットの削除

ファイル・シーケンス番号を利用しないテープ・データセットを削除する方法は、DASDに存在するデータセットを削除する方法と同じです。ここでは、ファイル・シーケンス番号を利用する場合のデータセットの削除方法について説明します。

テープ・デバイスでは新規データセットまたは既存のデータセットに新規レコードを作成する場合、その後ろにあるデータセットは破棄される特徴があります。OpenFrameでもこのような特徴をサポートします。テープの物理的な特徴によって削除されるのではなく、テープ・データセットがオープン(OUTPUTモード)されるときに、現在のデータセット以降のファイル・シーケンス番号のデータセットはすべて削除するように処理します。ただし、データセットがカタログに登録されている場合、カタログにはそのまま残ります。

テープ・データセットは、ジョブのDISPパラメータで削除することはできません。メインフレームではテープ・データセットをDISPパラメータでDELETEすることを禁じています。OpenFrameではDELETEを指定することは可能ですが、内部的にKEEPに変換されるため削除されません。ただし、dsdeleteツールを使用して削除することは可能です。dsdeleteツールを使ってデータセットを削除する場合、指定したデータセットとそれ以降のデータセットがすべて削除されます。

2.8.3. 注意事項

以下は、テープ・ボリウムを使用する際の注意事項です。

- テープ・ボリウムのロック

OpenFrameではファイル・シーケンス番号を利用する場合に限って、テープ・ボリウムを保護するためにデータセット・ロック以外に、ボリウム・ロックをかけられます。したがって、複数のジョブで同時に1つのテープ・ボリウムを使用することはできません。ただ、現在OpenFrameが提供するロックの特徴のため、同じアプリケーション・プログラムから1つのテープ・ボリウムにある複数のデータセットを同時に使用することを防ぐことはできません。このような場合は、データセットを使用するときに問題が発生することがあるので注意が必要です。

- DCB Handling APIの使用

ファイル・シーケンス番号を利用する場合は、DCB Handling APIを使用する際に注意が必要です。ファイル・シーケンス番号を利用する場合、テープ・データセット用のDCBはINPUTモード用に作成されます。OUTPUTモードでしようとするテープ・データセットは、オープンするまで正しい情報を持っていない可能性があります。したがって、DCB情報と属性の取得(Get Attributes)に関する関数は、テープ・データセットをオープンした後に実行してください。

第3章 VSAMデータセット

本章では、VSAMデータ形式、VSAMデータセットの種類、代替索引とスフィアおよびVSAMデータセットの管理方法について説明します。

3.1. 概要

1970年代に、IBMは3つのデータセット構造（順次・索引・直接アクセス）を統合した新しいアクセス方式と関連ユーティリティ・プログラムを開発しました。この新しいアクセス方式は、その当時、仮想記憶オペレーティング・システム(Virtual Storage Operating System)であるOS/VS1、OS/VS2と関連付けられ、VSAM(Virtual Storage Access Method、仮想記憶アクセス方式)と呼ばれることになります。

VSAMは、データの格納や検索に使用される技術であるアクセス方式の1つです。VSAMは、GET/PUTインターフェースを使用して、ディスク・デバイスとアプリケーション間でデータを送受信します。すべてのVSAMデータセットはカタログに登録される必要があり、テープ・デバイス・ボリュームには格納できません。

OpenFrameでは、VSAMに対応する製品としてTSAMをサポートしています。TSAMは、下部ストレージ技術としてデータベースを使用します。TSAMクライアントとサーバーの間では別のTSAMプロトコルを使用しているため、SQLオーバーヘッドがありません。また、大量のバッチ処理のための適切な通信プロトコルを実装するなど、性能向上を図っています。

3.2. VSAMデータ形式

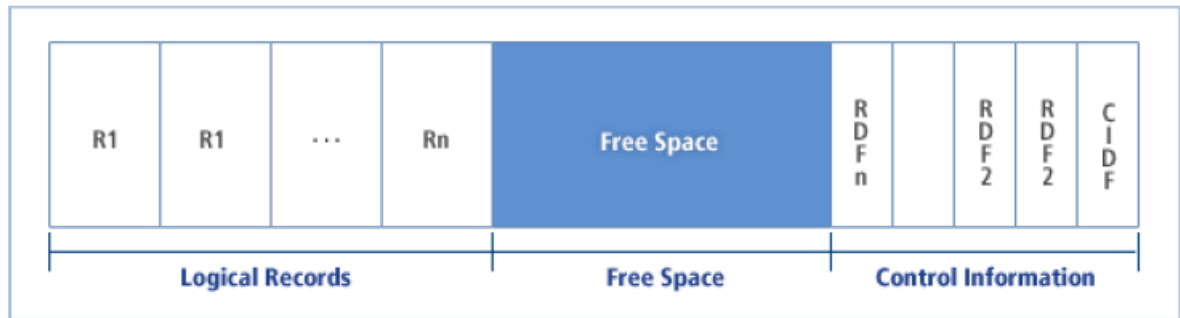
VSAMデータセットに保存されるデータ構造は、論理レコード単位でソートされます。論理レコードは、VSAMインターフェースを介してユーザーに渡される、ユーザー・レコードです。

VSAMデータセットでの論理レコードの格納方式は、非VSAMデータセットとは異なります。VSAMは、レコードを制御インターバル(Control Interval)に格納します。制御インターバルとは、VSAMがデータ・レコードとレコード制御データを格納するディスク・デバイスの連続的な領域です。VSAMレコードを読み取るときは、そのレコードが含まれている制御インターバル全体をメモリにあるVSAM入出力バッファーに読み込みます。

制御インターバル

各VSAMデータセットの制御インターバルのサイズはそれぞれ異なりますが、1つのデータセットにあるすべての制御インターバルのサイズは同じです。アクセス方式サービスのDEFINEコマンドを使用して、VSAMデータセットの制御インターバルのサイズを指定することができます。

[図 3.1] 制御インターバル



制御インターバルは以下の要素で構成されています。

- 論理レコード(Logical Records)
- 空き領域(Free Space)
- 制御情報(Control Information)

制御情報は、1つのCIDF(Control Interval Definition Field：制御インターバル定義フィールド)と、1つまたはそれ以上のRDFs(Record Definition Fields：レコード定義フィールド)から構成されます。CIDFは4バイトであり、空き領域のサイズと位置が記録されます。RDFsは3バイトであり、各レコードのサイズ・データが格納されます。

以下は、制御インターバルで使用する主要用語についての説明です。

用語	説明
制御領域(Control Area)	VSAMデータセットの連続的な制御インターバルをまとめて制御領域といいます。実際、VSAMデータセットは1つまたはそれ以上の制御領域から構成されています。1つの制御領域内にある制御インターバルの数は、VSAMによって決定されます。
スプリット(Splits)	空き領域がない制御インターバルにレコードを追加する場合、該当する制御インターバルではスプリットが発生します。該当する制御インターバルに格納されていたレコードの半分程度が別の制御インターバルに移され、新規レコードは元の制御インターバルに追加されます。
スパン・レコード (Spanned Records)	スパン・レコードは、制御インターバルのサイズより大きいレコードです。VSAMデータセットを定義するときに、SPANNEDパラメータを使用して1つのレコードが複数の制御インターバルにまたがって格納されるようにします。 データ・レコードのサイズが頻繁に変更されるデータセットや、平均レコードのサイズが制御インターバルのサイズより大きいデータセットでスパン・レコードを使用すると、ディスク容量の無駄な使用を軽減することができます。

用語	説明
コンポーネント	VSAMデータセットを構成する個別パートをコンポーネントといいます。 データ・コンポーネントと索引コンポーネントの2種類があります。 KSDS/VRDSは、データ・コンポーネントと索引コンポーネントを持っており、ESDS/RRDSは、データ・コンポーネントのみを持っています。
クラスター	クラスターは、最大2つで構成された関連コンポーネントの集合です。すべてのVSAMデータセットはクラスターとして定義されます。KSDSは、1つのクラスターは、1つのデータ・コンポーネントと1つの索引コンポーネントから構成されます。 RRDS/ESDSは、索引コンポーネントがないクラスターとみなされますが、一貫性を保つために、データセットを処理するときにクラスター名が使用されます。

OpenFrameでは、TSAMの下部ストレージの技術としてリレーショナル・データベースを使用しているため、前述の制御インターバルや制御領域などの技術は使用されません。

以下は、OpenFrame TSAMでVSAMデータセットをサポートするために使用されるデータベースの要素です。

Mainframe VSAM	OpenFrame TSAM	説明
ストレージ・ボリューム	表領域	ボリュームを表領域にマッピング
クラスター	表	クラスターを1つの表にマッピング
レコード	行	1レコードを1行にマッピング

3.3. VSAMデータセットの種類

VSAMは、索引、レコード番号、あるいは相対アドレスの順にデータ・レコードを格納、処理します。また、固定長レコードと可変長レコードに対する順次処理と直接処理も行うことができます。

VSAMは、以下の4つの方式でデータセットを構成します。

方式	説明
KSDS (Key Sequenced Data Set、キー順データセット)	キー順のデータセットです。
ESDS (Entry Sequenced Data Set、入力順データセット)	入力順のデータセットです。
RRDS (Relative Record Data Set、相対レコード・データセット)	レコード番号別のデータセットです。 RRDSは、レコードの長さによって以下の2つに分けられます。 – RRDS(Fixed-length Relative Record Data Set): 固定長RRDS

方式	説明
	– VRDS(Variable-length Relative Record Data Set): 可変長RRDS
LDS (Linear Data Set、線形データセット)	線形データセットです。 TSAMではLDSはサポートしていません。 日立メインフレームでは、LDS形式をFDS(Flat Data Set)形式に分類します。

3.3.1. KSDS (キー順データセット)

KSDS(Key Sequenced Data Set)のレコードは、レコードのキー順にデータセットに格納されます。

KSDSのキーは、レコードの特定のフィールドであり、以下の特性があります。

- すべてのレコードのキーの長さと位置は同じです。
- 同じキー値を持つ2つ以上のレコードは格納できません。
- キー値が格納された後は、レコードのキー値を変更できません。

KSDSは、固定長レコードと可変長レコードの両方をサポートします。新規レコードを追加する場合、キー値の昇順で適切な位置に挿入されます。

KSDSを効率的に管理するために、VSAMはデータセットにデータを格納するデータ・コンポーネントと、キー値の順にレコードに迅速なアクセスを可能にする索引コンポーネントを作成します。

3.3.2. ESDS (入力順データセット)

ESDS(Entry Sequenced Data Set)は、レコードが入ってくる順に格納されるという点で非VSAMデータセットと似ており、新規レコードはデータセットの最後に追加されます。

ESDSに格納されたレコードは削除できません。ただし、削除するレコード内容の一部に削除の印を付ける方法で、当該レコードをアプリケーションで論理的に削除することができます。

ESDSではレコードの更新は可能ですが、レコード長の変更はできません。レコード長の変更が必要な場合は、アプリケーション・レベルでレコード削除の印を付け、新しい長さのレコードをデータセットの末尾に追加する方法を使用します。ただし、データセットが使用されるアプリケーションでこのような処理が頻繁に行われることが予想される場合は、最初からKSDSを使用することをお勧めします。

ESDSは、索引コンポーネントは含まれていませんが、代替索引を使用することは許可されています。代替索引については、[「3.5. 代替索引とスフィア」](#)を参照してください。

3.3.3. RRDS（相対レコード・データセット）

固定長レコードのためのRRDS(Relative Record Data Set)は、レコードを事前に区画されたスロットに格納します。各スロットには番号が付けられ、レコードはレコードに付けられた番号に該当するスロットに格納されます。RRDSの1つのデータ・ブロックには、複数の固定長スロットが作成されます。

データ・ブロックのサイズは、（レコードの固定長 x 1ブロックに格納できるレコード数 + ブロック管理に使用されるオーバーヘッドのサイズ）のように指定してください。そうでない場合、RRDSの内部に使用できないスペースが生じます。ただし、このスペースは1つの固定長レコードより大きくなることはないため、クリティカルな問題が生じることはありません。

RRDSは固定長でレコードを格納するため、レコード番号が分かれば、簡単な計算でそのレコードの位置を検索でき、データ・レコードに効率的にアクセスすることができます。ただし、格納されているレコードの長さが固定されている点や、レコード番号でデータ・レコードを区別するという概念が、アプリケーションのドメインに適していない場合が多いため、管理するオブジェクト数が限定されている特別な場合に限って使用されます。

3.3.4. VRDS（可変長データセット）

VRDSは、可変長レコードを格納するRRDSです。ユーザーの観点からは、VRDSとRRDSは同じ方式で使用できます。すなわち、使用できるアクセス方式とレコードの指定に使用される引数が同じです。しかし、VSAMの内部から見ると、VRDSデータセットは、KSDS（キー順データセット）のように、索引コンポーネントとデータ・コンポーネントを使用してレコードを管理しています。

索引コンポーネントが必要な理由は、レコードの長さが固定されていないため、レコード番号だけではレコードが格納されている位置を容易に検索できないためです。一方、KSDSの管理と異なる点は、KSDSは索引コンポーネントのキーにレコードの一部であるキー領域を使用するのにに対し、VRDSではレコードとは別にユーザーが提供するレコード番号を索引コンポーネントのキーとして使用しています。索引レコードの索引情報は、KSDSと同じレコードの先頭アドレスの位置(相対バイト・アドレス：RBA)を使用します。

参考

OpenFrameではRRDSで固定長・可変長レコードをすべて処理するので、VRDSはサポートしません。

3.4. VSAMデータセット名

VSAMデータセットは、クラスター名とコンポーネント名を指定して作成することができます。

OpenFrameシステムでVSAMデータセットを作成するとき、クラスター名のみ指定し、コンポーネント名を指定しない場合は、コンポーネントの命名規則に従って名前を作成します。

コンポーネント名は次の3つの規則に従って指定されます。

- クラスターの最後の名前セグメントが「CLUSTER」の場合

- 最後の名前セグメントをデータ・コンポーネントは「DATA」に変更
- 索引コンポーネントは「INDEX」に変更

以下の例は、クラスターの名前が「TMAX.TEST.KSDS01.CLUSTER」の場合に、規則に従って生成されたデータ・コンポーネント、索引コンポーネントの名前を示します。

```
クラスター名      : TMAX.TEST.KSDS01.CLUSTER
データ・コンポーネント : TMAX.TEST.KSDS01.DATA
索引コンポーネント : TMAX.TEST.KSDS01.INDEX
```

- クラスター名の長さが38字より小さいか等しい場合

- データ・コンポーネントは「.DATA」を追加
- 索引コンポーネントは「.INDEX」を追加

以下の例は、クラスターの名前が「TMAX.TEST.KSDS01.INFO」の場合に、規則に従って生成されたデータ・コンポーネント、索引コンポーネントの名前を示します。

```
クラスター名      : TMAX.TEST.KSDS01.INFO
データ・コンポーネント : TMAX.TEST.KSDS01.INFO.DATA
索引コンポーネント : TMAX.TEST.KSDS01.INFO.INDEX
```

- クラスター名の長さが39字より大きい等しい場合

- データ・コンポーネントは「.D」を追加
- 索引コンポーネントは「.I」を追加

以下の例は、クラスターの名前が「TMAX.TEST.KSDS01.RESOURCE.FROM.REGION01」の場合に、規則に従って生成されたデータ・コンポーネント、索引コンポーネントの名前を示します。

```
クラスター名      : TMAX.TEST.KSDS01.RESOURCE.FROM.REGION01
データ・コンポーネント : TMAX.TEST.KSDS01.RESOURCE.FROM.REGION01.D
索引コンポーネント : TMAX.TEST.KSDS01.RESOURCE.FROM.REGION01.I
```

参考

クラスターの名前の長さが43~44字の場合、3番目の規則に従ってコンポーネントの名前の長さは45~46字になり、データセット名の長さ制限エラーが発生します。したがって、VSAMデータセットの名前はクラスター名のみ指定し、コンポーネントの名前は指定しない場合は、42文字に制限します。

3.5. 代替索引とスフィア

本節では、代替索引とスフィアについて記述します。

3.5.1. 代替索引

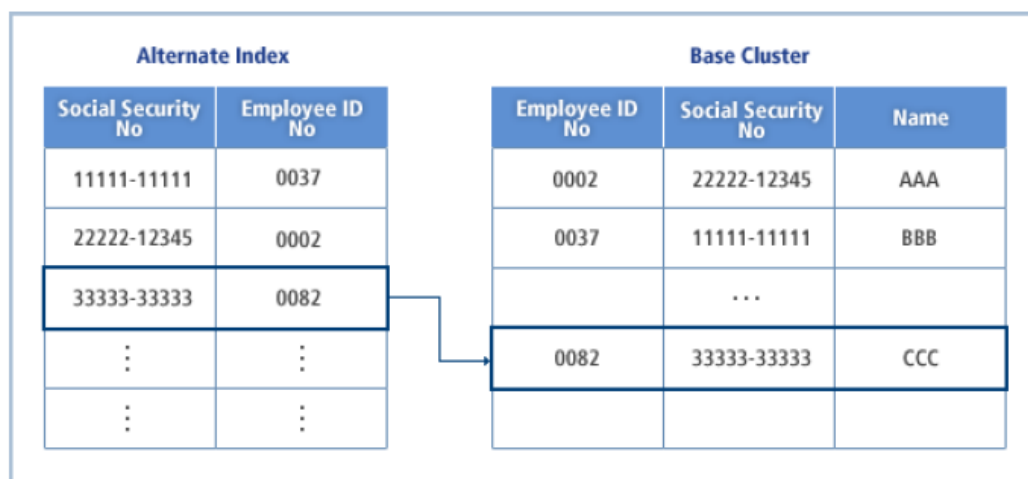
アクセス方式サービスを使用して、KSDSとESDS(基本クラスター)に代替索引(AIX:Alternate Index)を定義できます。代替索引は、基本クラスター・レコードに基本キー以外のキー・フィールドでアクセスする方法を提供します。

代替索引を使用すると、同じ情報を持つデータセットをアプリケーションごとに複数コピーして管理する必要がありません。

基本クラスター・レコードのどの部分も代替索引のキーとして使用できます。さらに、基本レコードの同じ部分を代替キーとして使用する代替索引を複数作成することも可能です。代替キーは、基本クラスター自体の基本キーとは異なり、1つのデータセット内のレコード間で一意である必要はありません。つまり、1つの代替キーに複数のレコードが対応できますが、OpenFrameではサポートしていません。

代替索引は、事実索引コンポーネントとデータ・コンポーネントから構成される独立したKSDSです。代替索引のレコードは、代替キーと基本クラスターのデータを指すポインター情報を含んでいます。OpenFrameでは、独立したKSDSとしてではなく、基本クラスターのテーブルにデータベースの索引を作成する方法で処理します。

[図 3.2] 代替索引



代替索引のアクセス・パス

代替索引を使用してKSDSやESDSを処理する前に、まずアクセス・パスを定義する必要があります。アクセス・パスは、代替索引を使用して基本クラスターを処理するときに使用されるカタログ・エントリです。

アクセス・パスは、IDCAMS(JSCVSUT/KQCAMS)ユーティリティのDEFINE PATHコマンドを使用して定義します。代替索引を使用して基本クラスターを処理するためには、各代替索引に対して少

なくとも1つ以上のアクセス・パスが定義されている必要があります。アクセス・パスは概念的に、基本クラスターと代替索引のペアとみなされます。

アプリケーションがパスを使用すると、基本クラスターと代替索引データセットと一緒に開かれます。基本クラスターとそれに関連したすべての代替索引の集合を**スフィア(Sphere)**といいます。

3.5.2. スフィア

スフィア(Sphere)は、基本クラスターと基本クラスターの代替索引の集合です。基本クラスターの代替索引は、1つの代替キーと多数の基本キーを持つレコードから構成されるKSDSクラスターです。

スフィア概念を導入すると、ベースとなるデータセットへのアクセス方式が追加されます。順次アクセス方式や直接アクセス方式といった区別ではなく、基本データセットの基本キー以外のキーを使用したアクセス方式です。これを、データへの複数のアクセス・パスが存在するといいます。

代表的な例として、COBOLアプリケーションのALTERNATE KEY文によるデータセットの使用を挙げることができます。VSAMアクセス方式は、KSDSやESDSに対し複数の代替キーを使用した順次および直接アクセス方式を提供します。このようなアクセス方式を、アクセス・パスを使用したアクセス方式といいます。

3.6. VSAMデータセットの管理

VSAMデータセットは、アクセス方式サービスのコマンドを使用して定義します。

以下は、VSAMデータセットを管理する方法についての説明です。

1. すべてのVSAMデータセットは、カタログに登録する必要があります。新規カタログを使用したい場合は、アクセス方式サービスのDEFINE USERCATALOGコマンドを使用して新規カタログを作成します。
2. VSAMデータセットは、アクセス方式サービスのDEFINE CLUSTERコマンドを使用してカタログに定義するか、JCLのDISP、RECORDパラメータを使用してVSAMデータセットを作成することができます。
3. アクセス方式サービスのREPROコマンドの使用するか、別のユーザー・プログラムを作成して、VSAMデータセットにデータをロードします。
4. 選択的に代替索引を基本クラスターに関連付けて使用したい場合は、アクセス方式サービスのDEFINE ALTERNATEINDEX、DEFINE PATH、BLDINDEXコマンドを使用します。

前述した全段階でVSAMデータセットの作業結果を確認するには、アクセス方式サービスのLISTCATまたはPRINTコマンドを使用します。LISTCATまたはPRINTコマンドは、VSAMデータセットに関連する問題を見つけ、修正するときに有用です。

クラスター概念

KSDSの場合、クラスターはデータ・コンポーネントと索引コンポーネントの組み合わせです。クラスターは、索引コンポーネントとデータ・コンポーネントを、まるで1つのコンポーネントであるかのように扱うことができます。また、固定長RRDSやESDSの場合は、索引コンポーネントなしで、データ・コンポーネントだけでクラスターとみなされますが、これは他のデータセットとの統一性を考慮してデータセットを処理するためです。

VSAMデータセットの定義

アクセス方式サービスのDEFINE CLUSTERコマンドを使用して、VSAMデータセットを定義することができます。クラスターを定義する際、クラスターを記述するために以下のカタログ・エントリが登録されます。

- クラスターを1つのコンポーネントであるかのように使用するためのクラスター項目
- クラスターのデータ・コンポーネントを記述するためのデータ項目
- KSDSの場合、クラスターの索引コンポーネントを記述するための索引項目

クラスターのすべての属性がカタログに記録されます。カタログには、データセットの管理やレコードの入出力のためのすべての情報が含まれます。

アクセス方式サービスを使用してVSAMデータセットを定義する場合、以下のようなパラメータを指定できます。

パラメータ	説明
INDEXED NOINDEXED NUMBERED	VSAMデータセットの種類を指定します。 – Key-Sequenced（キー順） – Entry-Sequenced（入力順） – Relative Record（相対レコード）
RECORDSIZE	平均または最大データ・レコードの長さを指定します。固定長データセットの場合、平均レコード長と最大レコード長を同じく指定します。
KEYS	KSDSでレコード内のキーの位置と長さを指定します。
CATALOG	クラスターが定義されるカタログの名前を指定します。
VOLUMES	クラスターのための領域が割り当てられるボリューム・シリアル番号を指定します。
RECORDS KILOBYTES MEGABYTES TRACKS CYLINDERS	クラスターのためのディスク領域のサイズを指定します。

パラメータ	説明
CONTROLINTERVALSIZE	VSAMが使用する制御インターバルの大きさを指定します。 OpenFrameではサポートしていません。
SPANNED	レコードが制御インターバルにまたがって格納できるかどうかを指定します。OpenFrameではサポートしていません。

また、JCLのDISPパラメータとRECORDGパラメータを使用してVSAMデータセットを定義することができます。以下は、JCLを使用してKSDSを定義する例です。

```
//DD1DD DSN=NEW.DATASET, DISP=(NEW, KEEP, DELETE), RECORDG=KS, LRECL=80
```

VSAMデータセットのロード

VSAMデータセットが定義されると、ソース・データセットから定義済みのVSAMデータセットにレコードをロードすることができます。ロードされるVSAMデータセットの種類によって、ソース・データセットは特定の順にソートされている必要があります。

- ESDSにロードされるデータ・レコード

特に決まった順序はありません。レコードの特定フィールドではなく、レコードが追加された順にソートされます。

- KSDSにロードされるデータ・レコード

入力データセットに重複キーがなく、昇順にソートされている必要があります。

- RRDSにロードされるデータ・レコード

昇順にソートされている必要があります。順次にロードされる場合は、VSAMがロードされる順に相対レコード番号を与えます。

データセットのロードは、以下の2つの方法で行います。

- REPROを使用してデータセットをロード

アクセス方式サービスのREPROコマンドを使用すると、順次データセットやVSAMデータセットからレコードを抽出し、VSAMアクセス方式のすべてのデータセットにロードすることができます。

- プログラムを作成してデータセットをロード

プログラムを作成してKSDSにロードするためには、まず、データ・レコードをキー順にソートした後、順次に格納します。

代替索引の定義

代替索引は、関連する基本レコードの代替キー順に索引項目を格納している、もう1つのKSDSです。代替索引は、クラスターのデータ・コンポーネントにあるレコードを検索する、また別の方法を提供します。

代替索引は、KSDSクラスターとESDSクラスターに対し定義することができます。代替索引を構成する手順は、以下のとおりです。

1. DEFINE CLUSTERコマンドを使用して、基本クラスターを定義します。
2. REPROコマンドを使用するか、プログラムを作成して、基本クラスターをロードします。
3. DEFINE ALTERNATEINDEXコマンドを使用して、代替索引を定義します。
4. DEFINE PATHコマンドを使用して、代替索引を基本クラスターに関連付けます。
5. BLDINDEXコマンドを使用して、代替索引を構成します。

参考

PATHを定義して代替索引を基本クラスターに関連付け、BLDINDEXコマンドで代替索引を構成することは、OpenFrameでは必須ではありません。

VSAMデータセットでは、代替索引を記述するために次の3つのカタログ・エントリを使用します。

- 代替索引をKSDSに記述する代替索引項目
- 代替索引のデータ・コンポーネントを記述するデータ項目
- 代替索引の索引コンポーネントを記述する索引項目

一意でないキーを持つ代替索引を定義する際には、レコード・サイズをなるべく大きく指定して、すべての一意でないキーが処理されるようにする必要があります。同じ代替キーを持つすべてのレコードの基本キー値は、1つの代替索引レコードに格納されます。

代替索引の管理

VSAMは、代替索引が常に基本クラスターと同期化されているとみなしているため、データセットを開く際に毎回同期化をチェックしません。

代替索引にUPGRADE属性を定義した場合、代替索引と関連付けられた基本クラスターが変更されると、代替索引も一緒に変更されます。この場合、基本クラスターが出力モードで開かれると、基本クラスターに関連付けられたすべての代替索引も自動的に開かれます。

OpenFrameでは基本クラスター・テーブルにデータベース索引を作成する方法で代替索引を作成するので、UPGRADE/NOUPGRADE属性はサポートしませんが、基本的な動作はUPGRADE属性を指定した場合と同じです。

アクセス・パスの定義

代替索引を定義した後、DEFINE PATHコマンドを使用して基本クラスターと代替索引との関係を定義する必要があります。アクセス・パスは、基本クラスターと代替索引のペアを示します。アクセス・パスを使用してデータセットにアクセスする場合、JCLのDSNAMEパラメータにアクセス・パス名を記述します。

ユーザー・プログラムからアクセス・パスでデータを処理する場合、代替索引と基本クラスターが同時に開かれます。基本クラスターのデータは、代替索引のキーを使用して読み取り/書き込みが行われます。

カタログ・エントリのリスト

VSAMデータセットが定義された後、アクセス方式サービスのLISTCATコマンドを使用すると、カタログに登録されている項目の一部あるいは全部を確認できます。

LISTCATコマンドは、カタログに登録されている次の情報を表示します。

- SMS属性を含む全てのオブジェクトの属性
- 作成日と有効期限
- データセットの構造
- データセットの使用統計
- スペースの割り当て情報およびボリューム情報

データセットのコピー

様々な理由により、データセットをコピーしたり、2つのデータセットを1つにまとめたりする場合があります。たとえば、テスト用のコピー・データセットを作成する場合や、他の目的で2つのコピーを作成する場合です。また、データセットを更新する前にバックアップ用としてコピーしておく必要もあります。このような場合、アクセス方式サービスのREPROコマンドを使用してデータセットをコピーすることができます。

以下のような場合に、REPROを使用してデータセットをコピーすることができます。

- VSAMデータセットをコピーして新規VSAMデータセットを作成する場合
- 順次データセットをコピーして新規順次データセットを作成する場合
- 順次データセットをコピーしてVSAMデータセットに変換する場合
- VSAMデータセットをコピーして順次データセットに変換する場合
- VSAMデータセットをコピーして自動的に再編成をする場合
- PDSやPDSE(Partitioned DataSet Extended)などの各メンバーをコピーする場合

KSDSにコピーする場合、入力データセットのデータ・レコードは重複するキーなしで、昇順にソートされている必要があります。

VSAMデータセットがキー順にコピーされる場合、以下のような再構成(reorganization)が自動的に実行されます。

- 論理レコードの物理的な位置ソート
- データセットの内部的な空き容量の分配
- VSAM索引コンポーネントの再構成

データセットの出力

データセットの使用中に問題が生じた場合、アクセス方式サービスのPRINTコマンドを使って、VSAMデータセットの一部あるいは全レコードを出力することができます。

この際、論理レコードのデータ内容のみ出力され、システムが管理している制御フィールドは出力されません。出力された各レコードは、以下のいずれかの方法で区別されます。

- ESDSの場合は、相対バイト・アドレス(RBA)
- KSDSの場合は、キー値(KEY)
- RRDSの場合は、レコード番号(RRN)

データセットの削除

アクセス方式サービスのDELETEコマンドを使用すると、データセットまたはカタログを削除することができます。DELETEコマンドは、ボリュームからデータセットを削除し、カタログからデータセットの項目を削除します。

ブラウジング

以下は、ブラウジングをサポートするインターフェースです。

インターフェース	説明
START_BR	TSAMで任意のレコードを読み取るときと同じ方法でブラウジングを始めます。 ブラウザーのポインターを初期化するだけで、レコードは読み取りません。
READ_NEXT	START_BRインターフェースで指定した開始位置からレコードを順次読み取ります。
READ_PREV	データセットを逆順に読み取ります。

インターフェース	説明
RESET_BR	ブラウジングが始まったら、現在ブラウジングするポインターを変更します。 READ_NEXTまたはREAD_PREVインターフェースのパラメータのRIDFLDを変更しても、ブラウジング・ポインターが変更されます。

トランザクション

トランザクションは、一度に処理しなければならない1つ以上の一連の作業のグループを表す論理的な作業単位です。

以下は、トランザクションをサポートするインターフェースです。

インターフェース	説明
COMMIT	現在のトランザクションをコミットします。
ROLLBACK	現在のトランザクションをロールバックします。

中間コミット

OpenFrameのTSAMはRDBのテーブルとして実装されているため、データをテーブルに反映するためにはデータベースにコミットを要求する必要があります。OpenFrameでは、バッチ処理などによって更新するデータがたまりすぎてパフォーマンスが低下する問題を防ぐために、1024個のコマンドごとにデータをデータベースにコミットし、ジョブが正常に終了したときに残りのデータをコミットします。

そのため、ジョブが実行中に異常終了しても、中間コミットされたデータはロールバックされずにデータベースに適用されるため、データセットのステータスはジョブが実行される前と異なることがあります。また、ホスト・システムのVSAMは、非VSAMと同様に変更内容がデータベースに即時反映されるため、OpenFrameの動作と差があります。

第4章 統合カタログ

本章では、カタログの構造、検索順、マスター・カタログなど、カタログについての全般的な事項およびカタログの構造と管理方法について記述し、最後に、カタログ関連のツール・プログラムについて説明します。

4.1. 概要

OpenFrameでは、データセットのメタ情報を管理するために、総合カタログ・モジュールと関連ツール・プログラムを提供しています。カタログは、OpenFrameデータセットのメタ情報の窓口としての役割をします。

VSAM(仮想記憶アクセス方式)データセットを使用するには、必ずデータセットがカタログに登録されている必要があります。非VSAM(非仮想記憶アクセス方式)データセットは、カタログに登録されていなくても使用できます。カタログに登録されていない非VSAMデータセットを使用する場合は、JCLにデータセットのすべての情報またはボリュームを指定する必要があります。

4.2. カタログ

カタログには、他のデータセットのメタ情報が格納されています。カタログを使用すると、データセットの具体的な格納位置が分からなくても、名前だけでデータセットを使用することができます。データセットをカタログ化することにより、システム・ストレージ設定についての知識がなくても、データセットを使用することができます。

4.2.1. カタログの構造

メインフレームでのカタログは、BCSデータセットとVVDSデータセットという2種類の異なるデータセットで構成されています。一方、OpenFrameでは、BCSデータセットはカタログ表として、VVDSデータセットはVTOCと統合されVTOC表として構成されます。

カタログ表には、データセットの基本属性、ボリューム情報、使用権限、別の項目との関連付けなどの情報が含まれます。

4.2.2. カタログの検索順

ユーザーがカタログに登録されたデータセットを使用する場合や、カタログに登録するデータセットを作成する際、適切なカタログを見つけるために、複数のカタログを対象として検索が行われます。

カタログの検索順は、カタログに定義されているエイリアスやカタログ名、エイリアスの検索レベルによって決まります。エイリアスの検索レベルは、データセット名の上位修飾子を何個まで検索するかを決定するシステム設定です。

ほとんどのカタログ検索は、カタログのエイリアスに基づいています。すべてのカタログに適切なエイリアスが定義されていれば、データセット名の上位修飾子が特定のカタログのエイリアスと一致することになるため、ユーザーの要求を処理する適切なカタログを簡単に見つけることができます。

前述の方法以外に、JCLでJOB CATまたはSTEP CATを使用したり、アクセス方式サービスでCATALOGパラメータを使用したりしてカタログを指定する方法があります。

以下は、データセットの検索や定義に使用されるカタログの検索順です。

1. システム・カタログを指定した場合は、システム・カタログを検索します。
2. IDCAMS(JSCVSUT/KQCAMS)ユーティリティのCATALOGパラメータで指定されたカタログを検索します。
3. GDSデータセットの場合、GDG BASEが定義されているカタログを検索します。
4. JCLでSTEP CATまたはJOB CATに指定されたカタログを検索します。
5. 上位修飾子が特定のカタログのエイリアスである場合、そのカタログを検索します。
6. 最後にマスター・カタログを検索します。

参考

OpenFrameでは、システム・カタログを使用しても、JCLでAMPパラメータにAMORGを指定した場合は、システム・カタログを検索しません。

4.2.3. マスター・カタログ

マスター・カタログは、構造的にはユーザー・カタログと類似していますが、使用方法やカタログ化されるデータセットが異なります。OpenFrameシステムでは、一度に1つのマスター・カタログのみ使用できます。

マスター・カタログには、システムに登録されているすべてのユーザー・カタログと、ユーザー・カタログを指すエイリアス項目が登録されています。また、アプリケーション・プログラムで使用する一般的なデータセットを登録することも可能です。

マスター・カタログに登録される項目は、以下のとおりです。

カタログ・エントリ	説明
ユーザー・カタログ	システムに登録されているすべてのユーザー・カタログです。
エイリアス項目	システムに登録されているユーザー・カタログを指すエイリアス項目です。

カタログ・エントリ	説明
一般的なデータセット	OpenFrameの起動に必要なシステム表、ユーザー・カタログ、エイリアス以外の一般的なデータセットです。

4.2.4. システム・カタログ

システム・カタログは、統合カタログを使用するの前に使われていたカタログ・システムです。OpenFrameでは、ユーザー・カタログの1つをシステム・カタログに指定して使用できます。OpenFrameシステム内では、マスター・カタログと同様に、1つのシステム・カタログのみ使用できます。

システム・カタログを使用する場合は、非VSAMとGDG、GDS、非VSAMのエイリアスをシステム・カタログに登録し、VSAMについてはマスター・カタログやユーザー・カタログ・システムに登録して使用します。

また、カタログからデータセットを検索する際、システム・カタログを使用する場合は、そこから検索することになります。ただし、JCLのFD文でAMPパラメータに「AMORG」を指定した場合は、システム・カタログを検索しません。また、dscreate、dsdelete、gdgcreate、gdgdeleteなどのデータセット管理ツールを使用して新しい非VSAMまたはGDGを定義または削除する場合は、カタログ名を指定した場合でもシステム・カタログに登録または削除します。

参考

システム・カタログを使用するための設定については、『OpenFrame 環境設定ガイド』を参照してください。

4.2.5. カatalog・エントリ

以下は、OpenFrameの統合カタログに登録するエントリの種類と各エントリの説明です。

カタログ・エントリ	説明
Non-VSAM Data Set	非VSAMデータセットのカタログ・エントリです。 カタログに登録された非VSAMデータセットは、カタログからボリューム情報を取得することが可能であり、これを利用してVTOCからデータセット情報を取得します。
GDG	世代データ・グループのカタログ・エントリです。 GDG LIMITなどのGDGの属性情報と世代データ・グループに関連するGDSデータセット名が管理されます。
GDS	世代データセットのカタログ・エントリです。 基本的に格納されるデータの内容は、非VSAMデータセット項目と同じです。関連項目として世代データ・グループ名が格納されます。

カタログ・エントリ	説明
VSAM Cluster	VSAMクラスターのカタログ・エントリです。 VSAMデータ・コンポーネントと索引コンポーネントを1つの名前で管理するためにカタログに登録します。関連項目としてデータ・コンポーネントと索引コンポーネントの名前が格納されます。
Data Component	VSAMデータ・コンポーネントのカタログ・エントリです。 VSAMデータセットの実際のレコード・データが格納されるオブジェクトの項目です。
Index Component	VSAM索引コンポーネントのカタログ・エントリです。 VSAMデータセットの基本キーを迅速に検索するためのバイナリ・ツリー・オブジェクトの項目です。
Alternate Index	代替索引のカタログ・エントリです。 カタログで管理される情報の内容は基本的にVSAMクラスターと同じであり、項目の種類のみ異なります。関連項目としてデータ・コンポーネント名、索引コンポーネント名、基本クラスター名が格納されます。
Path Entry	代替索引パスのカタログ・エントリです。 代替索引を使用して基本クラスターへのアクセス・パスを定義します。関連項目として代替索引名と基本クラスター名が格納されます。
User Catalog	ユーザー・カタログのカタログ・エントリです。 ユーザー・カタログもマスター・カタログにカタログされる必要があります。格納される情報は基本的にVSAMクラスターと同様です。
Alias Entry	カタログ・エイリアスのカタログ・エントリです。 ユーザー・カタログには複数のエイリアス項目が登録でき、登録されたエイリアスを使ってカタログの検索順が調整されます。

4.2.6. カタログの情報

OpenFrameの統合カタログで管理する情報は、以下の情報グループに分類されます。この中で、Allocation GroupとStatistics Groupの情報はOpenFrameのシステム表で管理されます。

情報グループ	説明
History Group	項目名、項目の種類、作成日、有効期限、所有者IDが格納されます。
SMS Group	ストレージ・クラス、マネジメント・クラス、データ・クラスが格納されます。
Protection Group	セキュリティ設定とパスワードが格納されます。現在のOpenFrameでは使用されません。

情報グループ	説明
Volumes Group	ボリューム・シリアルとデバイスの種類が格納されます。
Attribute Group	レコードの平均サイズと最大サイズ、キーの位置と長さ、VSAMの種類と属性、AIXの属性と共有オプションが格納されます。
GDG Group	GDGの限度、GDGの属性、最終変更日が格納されます。
Non-VSAM Group	非VSAMデータセットの種類（PDS、GDS、その他の非VSAMデータセット）情報が格納されます。
Allocation Group	最初の領域サイズ、拡張領域サイズ、最大の使用領域および拡張回数、最終変更日が格納されます。
Association Group	関連項目数と項目名、項目の種類などが格納されます。

4.3. カタログの管理

ユーザー・カタログの定義

アクセス方式サービスのDEFINE USERCATALOGコマンドを使用すると、新規ユーザー・カタログを定義することができます。DEFINE USERCATALOGコマンドを使用する際は、ICFCATALOGパラメータを指定する必要があります。

以下は、ユーザー・カタログのSYS1.ICFCAT.TESTを、SYS305ボリュームに作成する例です。（以下の例のIDCAMSユーティリティは、製品に応じてJSCVSUT、KQCAMSユーティリティに代替できます。）

```
//DEFCAT   EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN    DD *
  DEFINE USERCATALOG                -
    ( NAME(SYS1.ICFCAT.TEST)        -
      MEGABYTES(15 15)              -
      VOLUME(SYS305)                -
      ICFCATALOG                    -
      FREESPACE(10 10)              -
      STRNO(3 )                    -
      DATA( CISC(4096)              -
        BUFND(4 )                  -
        INDEX( BUFNI(4 )            -
/*
```

カタログ・エイリアスの定義

システムがカタログを使用するためには、データセットを登録するカタログを決める必要があります。この際、カタログにエイリアスを定義しておくことで、容易に作業を進めることができます。

エイリアスの作成が正しくない場合、ユーザーがデータセットにアクセスできないことがあります。そのため、新しいエイリアスを定義する際は、既存のデータセットに与える影響を考慮して行う必要があります。

アクセス方式サービスのDEFINE USERCATALOGコマンドを使用してユーザー・カタログを定義した後、DEFINE ALIASコマンドを使ってカタログのエイリアスを定義することができます。カタログのエイリアスは、ユーザー・カタログと同様、マスター・カタログに定義される必要があります。

以下は、ユーザー・カタログのSYS1.ICFCAT.TESTに、USER0001とPROJECTAの2つのエイリアスを定義する例です。（以下の例のIDCAMSユーティリティは、製品に応じてJSCVSUT、KQCAMSユーティリティに代替できます。）

```
//ALIAS      EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN      DD *
  DEFINE ALIAS                                -
      ( NAME(USER0001)                        -
        RELATE(SYS1.ICFCAT.TEST) )
  DEFINE ALIAS                                -
      ( NAME(PROJECTA)                       -
        RELATE(SYS1.ICFCAT.TEST) )
/*
```

カタログ・エントリのリスト化

アクセス方式サービスのLISTCATコマンドを使用してカタログ・レコード・リストを出力し、カタログ・データセットの情報を確認することができます。

ただし、ユーザー・カタログのLISTCAT I/Oの統計情報は正確ではありません。

以下は、LISTCATを使用して特定のユーザー・カタログに関連したすべてのエイリアスを照会する例です。エイリアスは、Associations Groupセクションに表示されます。（以下の例のIDCAMSユーティリティは、製品に応じてJSCVSUT、KQCAMSユーティリティに代替できます。）

```
//LSTALIAS EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN      DD *
  LISTCAT ALL ENTRIES(SYS1.ICFCAT.TEST)
/*
```

ユーザー・カタログの削除

アクセス方式サービスのDELETEコマンドを使用すると、ユーザー・カタログやカタログされたデータセット、その他のすべてのカタログ・エントリを削除することができます。

一般的にユーザー・カタログを削除するには、ユーザー・カタログに登録されている項目があってはなりませんが、必要に応じてオプションを使ってカタログ・エントリが含まれているユーザー・カタログを削除することができます。

ユーザー・カタログが空の場合、DELETE USERCATALOGコマンドを使って削除することができます。このコマンドは、ユーザー・カタログに登録されているデータセット項目と、マスター・カタログに登録されているユーザー・カタログ・エントリを削除します。

ユーザー・カタログが空でない場合は、DELETE USERCATALOGコマンドの使用時にFORCEオプションを使用します。FORCEオプションを使用すると、ユーザー・カタログに関連するエイリアスが削除され、そのカタログに登録されているすべてのデータセットとカタログ・エントリが削除されます。

4.4. 関連プログラム

関連ツールまたはユーティリティには、IDCAMS(JSCVSUT/KQCAMS)とvolmgrがあります。

ツール/ユーティリティ	説明
IDCAMS(JSCVSUT/KQCAMS)	メインフレームのデータセットおよびカタログ情報を管理するユーティリティであり、VSAMおよび非VSAMデータセットの両方を管理できます。詳細については、『OpenFrame ユーティリティリファレンスガイド』を参照してください。
volmgr	OpenFrame製品のインストール時に、ボリュームを使用するために必要な管理領域を作成します。詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。

付録 A. カタログ管理API

本付録では、カタログ管理APIを使用してカタログ・エントリとデータセットを管理する方法について説明します。

A.1. 概要

OpenFrameが提供するカタログ管理APIを使用して、カタログ・エントリの削除・検索など、カタログとデータセットを管理することができます。

カタログ管理APIを使用してプログラムを作成するには、OpenFrameバイナリと一緒に配布される `amsu.h` ヘッダー・ファイルをユーザー・プログラムに含める必要があります、かつ、ユーザー・プログラムをコンパイルする際、`libams.so` ライブラリをリンクする必要があります。

カタログ管理APIは、以下のように分類できます。

● 初期化と終了 (Initialization & Finalization)

API	説明
<code>amsu_initialize()</code>	カタログ管理のための初期化を行います。
<code>amsu_finalize()</code>	カタログ管理を終了する処理を行います。

● ボリューム情報 (Volume Information)

API	説明
<code>amsu_volume_list()</code>	システムに登録されているボリューム・リストを取得します。
<code>amsu_default_volume()</code>	システムに登録されているデフォルト・ボリュームを取得します。
<code>amsu_volume_path()</code>	特定のボリュームのディレクトリ・パスを取得します。

● カタログ検索 (Catalog Search)

API	説明
<code>amsu_use_catalogs()</code>	任意のカタログ検索順を指定します。
<code>amsu_search_entries()</code>	カタログから特定の項目を検索します。
<code>amsu_master_catalog()</code>	システムに登録されているマスター・カタログ情報を取得します。

API	説明
amsu_candidate_catalog()	特定のデータセットをカタログ化するための対象カタログを検索します。

- ファイル・パス検索 (File Path Resolution)

API	説明
amsu_filepath2()	特定のデータセットのUNIXファイル・パスを取得します。
amsu_filename2()	特定のデータセットのファイル名を取得します。

- エントリ管理 (Entry Management)

API	説明
amsu_delete()	カタログに登録されているデータセットを削除します。
amsu_info()	カタログに登録されている項目情報を読み取ります。
amsu_assoc()	カタログに登録されている項目と関連した一覧を取得します。

A.2. 初期化と終了

A.2.1. [amsu_initialize\(\)](#)

カタログ管理のための初期化を行います。カタログの管理に必要なリソースをメモリにロードします。この関数を呼び出さずに他のカタログ管理APIを使用するとエラーが発生するので、必ず初期化してください。

- プロトタイプ

```
int amsu_initialize();
```

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.2.2. [amsu_finalize\(\)](#)

カタログ管理を終了する場合に呼び出します。カタログ管理のためにメモリにロードしたリソースを削除します。

- プロトタイプ

```
int amsu_finalize();
```


- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.3. ボリューム情報

A.3.1. amsu_volume_list()

システムに登録されているボリューム・リストを取得します。

- プロトタイプ

```
int amsu_volume_list(int *count, char *volser[]);
```

- パラメータ

パラメータ	説明
count(出力)	volserに与えられたバッファ・サイズ(入力)と登録されたボリューム数です。
volser(出力)	登録されたボリューム・シリアルを保存するためのバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.3.2. amsu_default_volume()

システムに登録されているデフォルト・ボリュームを取得します。

- プロトタイプ

```
int amsu_default_volume(char *volser);
```

- パラメータ

パラメータ	説明
volser(出力)	デフォルト・ボリューム・シリアルを保存するためのバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.3.3. amsu_volume_path()

特定のボリュームのディレクトリ・パスを取得します。

- プロトタイプ

```
int amsu_volume_path(char *volser, char *vpath);
```

- パラメータ

パラメータ	説明
volser(入力)	ディレクトリ・パスを確認するボリューム・シリアルです。
vpath(出力)	ディレクトリ・パスを保存するためのバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.4. カタログ検索

A.4.1. amsu_use_catalogs()

任意のカタログ検索順を指定します。

- プロトタイプ

```
int amsu_use_catalogs(char **cat_names);  
int amsu_use_catalog(char *cat_name);
```

- パラメータ

パラメータ	説明
cat_names(入力)	カタログの検索時に優先して検索するカタログ名のリストです。
cat_name(入力)	カタログの検索時に優先して検索するカタログ名です。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.4.2. amsu_search_entries()

カタログで特定の項目を検索します。カタログの検索順は以下のとおりです。

1. amsu_use_catalogs()によって指定されたカタログ

2. filter_keyによって指定されたエイリアス・カタログ

3. 前項で見つからなかった場合はマスター・カタログ

- プロトタイプ

```
int amsu_search_entries(char *filter_key, char *entry_types, int *count,
                        amsu_result_t *result, int flags);
```

- パラメータ

パラメータ	説明
filter_key	検索するカタログ・エントリの式です。 (IN - 「*」 「%」などのワイルドカード文字を含む)
entry_types(入力)	検索するカタログ・エントリのタイプです。(NULLの場合はすべて検索)
count(出力)	結果バッファのサイズ(入力)と検索された項目の数です。
result(出力)	検索された項目を保存するバッファです。
flags(入力)	カタログの検索オプションです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

カタログ・エントリのタイプとカタログの検索オプションは、ams_user.hヘッダー・ファイルに定義されているマクロを参照します。

マクロ	項目タイプ	説明
AMSU_ENTRY_TYPE_NONVSAM	'A'	非VSAMです。
AMSU_ENTRY_TYPE_GDG	'B'	世代データ・グループです。
AMSU_ENTRY_TYPE_CLUSTER	'C'	VSAMクラスターです。
AMSU_ENTRY_TYPE_DATA	'D'	データ・コンポーネントです。
AMSU_ENTRY_TYPE_AIX	'G'	代替索引です。
AMSU_ENTRY_TYPE_GDS	'H'	世代データセットです。
AMSU_ENTRY_TYPE_INDEX	'I'	索引コンポーネントです。
AMSU_ENTRY_TYPE_PATH	'R'	アクセス・パスです。
AMSU_ENTRY_TYPE_UCAT	'U'	ユーザー・カタログです。
AMSU_ENTRY_TYPE_ALIAS	'X'	エイリアスです。

以下は、カタログ検索オプションのマクロです。

マクロ	検索オプション	説明
AMSU_SEARCH_DEFAULT	0x00	基本検索オプションです。
AMSU_SEARCH_CLS_COMPS	0x01	クラスター名がフィルター・キーと一致する場合、コンポーネントが返されます。
AMSU_SEARCH_1_CATALOG	0x02	1つだけのカタログが検索されます。

カタログの検索結果は、以下のような構造体に格納されます。

```
/* ----- catalog search result ----- */
typedef struct _amsu_result_t {
    char        catname[AMSU_ENTRY_NAME_LENGTH+1];
    char        entname[AMSU_ENTRY_NAME_LENGTH+1];
    char        enttype;
} amsu_result_t;
```

A.4.3. amsu_master_catalog()

システムに登録されているマスター・カタログの情報を取得します。

- プロトタイプ

```
int amsu_master_catalog(char *mascat_name, char *mascat_vser);
```

- パラメータ

パラメータ	説明
mascat_name(出力)	マスター・カタログのデータセット名を保存するバッファです。
mascat_vser(出力)	マスター・カタログのボリューム・シリアル番号を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.4.4. amsu_candidate_catalog()

特定のデータセットの対象カタログを検索します。

対象カタログは、以下の規則により決まります。

1. データセット名がGDSの場合はGDGカタログ
2. amsu_use_catalogs()によって指定されたカタログ

3. データセット名によって指定されたエイリアス・カタログ

4. 前項で決まらなかった場合はマスター・カタログ

- プロトタイプ

```
int amsu_candidate_catalog(char *dsname, char *cat_name);
```

- パラメータ

パラメータ	説明
dsname(入力)	カタログ化するデータセット名です。
cat_name(出力)	対象カタログの名前を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.5. ファイル・パス検索

A.5.1. amsu_filepath2()

特定のデータセットのファイル・パスを取得します。

- プロトタイプ

```
int amsu_filepath(char *dsname, char *volser, char *filepath);  
int amsu_filepath2(char *dsname, char *memname, char *volser, char *filepath);
```

- パラメータ

パラメータ	説明
dsname(入力)	ファイル・パスを検索するデータセット名です。
memname(入力)	ファイル・パスを検索するPDSのメンバーです。
volser(入力)	指定されたデータセットが格納されているボリューム・シリアルです。
filepath(出力)	UNIXファイル・パスを保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.5.2. amsu_filename2()

特定のデータセットのファイル名を取得します。

- プロトタイプ

```
int amsu_filename(char *dsname, char *filename);  
int amsu_filename2(char *dsname, char *membname, char *filename);
```

- パラメータ

パラメータ	説明
dsname(入力)	ファイル名を検索するデータセット名です。
membname(入力)	ファイル名を検索するPDSのメンバーです。
filename(出力)	UNIXファイル名を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.6. エントリ管理

A.6.1. amsu_delete()

カタログに登録されている項目を削除します。

- プロトタイプ

```
int amsu_delete(char *cat_name, char *entry_name, char entry_type, int flags);
```

- パラメータ

パラメータ	説明
cat_name(入力)	削除する項目が登録されたカタログ名です。
entry_name(入力)	削除する項目がカタログされた名前です。
entry_type(入力)	削除する項目がカタログされたタイプです。
flags(入力)	カタログの削除オプションです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

A.6.2. amsu_info()

カタログに登録されている項目の情報を読み取ります。

- プロトタイプ

```
int amsu_info(char *cat_name, char *entry_name, char entry_type, void
               *entry_info, int flags);
```

- パラメータ

パラメータ	説明
cat_name(入力)	読み取る項目が登録されたカタログ名です。
entry_name(入力)	読み取る項目がカタログされた名前です。
entry_type(入力)	読み取る項目がカタログされたタイプです。
flags(入力)	カタログの削除オプションです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

カタログ・エントリの情報は、項目タイプによってそれぞれ異なる構造体に格納されます。以下は、GDG項目の情報構造体を示しています。

```
/* ----- GDG info type ----- */

typedef struct _amsu_gdg_info_t {
    char    dscrdt2[8];    /* creation date */
    char    dsexdt2[8];    /* expiration date */
    char    ownerid[8];    /* owner of the data set */
    int16_t gdglimit;      /* maximum number of GDS allowed */
    uint8_t gdgattr;       /* GDG attributes - bit flags */
} amsu_gdg_info_t;
```

GDG以外の項目については、ams_user.hヘッダー・ファイルに定義されている構造体を参照してください。

A.6.3. amsu_assoc()

カタログに登録されている項目と関連した項目の一覧を取得します。

- プロトタイプ

```
int amsu_assoc(char *cat_name, char *entry_name, char entry_type, int
               *count, amsu_symbol_t *assocs);
```

- パラメータ

パラメータ	説明
cat_name(入力)	項目が登録されたカタログ名です。
entry_name(入力)	項目がカタログされた名前です。
entry_type(入力)	項目がカタログされたタイプです。
count(出力)	関連項目のバッファ長(入力)、関連項目の数です。
assoc(出力)	関連項目のバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

関連項目の一覧は、以下の構造体に格納されます。

```
/* ----- catalog entry symbol ----- */

typedef struct _amsu_symbol_t {
    char      entname[AMSU_ENTRY_NAME_LENGTH+1];
    char      enttype;
} amsu_symbol_t;
```


付録 B. データセットI/O API

本付録では、データセットI/O APIを使用して、C言語で作成されたアプリケーション・プログラムでデータセットのレコードを読み書きする方法について説明します。

B.1. 概要

OpenFrameが提供するデータセットI/O APIを使用して、C言語で作成されたアプリケーション・プログラムで、JCLのDD文で割り当てられたデータセットのレコードを読み書きすることができます。

参考

メインCOBOLアプリケーション・プログラムでオープンされたデータセットを、Cで作成されたサブプログラムで読み書きするなどのI/Oを行う場合は、FCD Handling APIライブラリを使用することをお勧めします。

データセットI/O APIを使用してプログラムを作成するには、OpenFrameバイナリと一緒に配布されるtcfh.hヘッダー・ファイルをユーザー・プログラムに含める必要があり、ユーザー・プログラムをコンパイルする際、libtcfh.soライブラリをリンクする必要があります。

データセットI/O APIは、以下のように分類できます。

● ファイルのオープンとクローズ (File Open/Close)

API	説明
tcfh_open()	レコード処理のためにデータセットを開きます。
tcfh_close()	レコード処理のために開いていたデータセットを閉じます。

● レコード・アクセス (Record Access)

API	説明
tcfh_start()	順次処理のために内部レコード・ポインターを移動させます。
tcfh_read()	データセットで1つのレコードを読み取ります。
tcfh_write()	データセットに1つのレコードを書き込みます。
tcfh_rewrite()	データセットにある1つのレコードの内容を変更します。
tcfh_delete()	データセットにある1つのレコードを削除します。

- PDSメンバーの処理 (PDS Member Handling)

API	説明
tcfh_bldl()	データセットのディレクトリー一覧を構成します。
tcfh_bldl_info()	データセットのディレクトリー一覧を検索します。
tcfh_find_by_bldl()	データセットのディレクトリー一覧を使用してメンバーにアクセスします。
tcfh_find_by_name()	データセットのメンバー名を使用してメンバーにアクセスします。
tcfh_stow()	データセット・メンバーの追加、取り替え、変更、削除、初期化を行います。

B.2. ファイルのオープンとクローズ

B.2.1. tcfh_open()

レコード処理のためにデータセットを開きます。データセットを開く操作は、JCLのDD文で割り当てられたデータセットに対して、実際にレコードを読み書きするなどのI/Oを行うための準備作業です。

- プロトタイプ

```
int tcfh_open(tcfh_file_t *file, int open_mode, int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	オープンするデータセットの情報を保存するファイル構造体です。
open_mode(入力)	データセットのオープン・モードです。 – INPUT – OUTPUT – INOUT – EXTEND
flags(入力)	データセットのオープン・オプションです。 – NO REWIND – VB MOVE : VBデータセットをオープンするとき、レコード長にRDWが含まれないようにします。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

データセットを開くためには、オープンするデータセットの情報を保存したファイル構造体のポインターを提供する必要があります。このファイル構造体は、tcfh.hヘッダー・ファイルに定義されています。

```
/* ----- file I/O block ----- */

typedef struct tcfh_file_s {
    char        file_name[8+2];    /* file name - SELECT clause */
    char        file_status[2];    /* file status - I/O interface */
    uint8_t     organization;      /* organization - SELECT clause */
    uint8_t     access_mode;       /* access mode - SELECT clause */
    uint8_t     open_mode;         /* open mode - tcfh_open() */
    uint8_t     misc_flags;        /* miscellaneous - SELECT clause */
    /* 64 bit separating line */
    char        file_path[256];    /* file path - tcfh_open() */
    /* 64 bit separating line */
    int32_t     file_handle;        /* file handle - tcfh_open() */
    int32_t     relative_key;       /* relative key - C(C++) program */
    /* 64 bit separating line */
    int16_t     key_length;         /* key length - DCL statement */
    int16_t     key_loc;           /* key position - DCL statement */
    int16_t     rec_size;          /* record size - DCL statement */
    int16_t     cur_reclen;        /* current record length - I/O interface */
    ...
} tcfh_file_t;
```

前述したファイル構造体で、データセットを開く際に指定が必要な項目は以下のとおりです。

項目	説明
file_name	JCLに記述されたDD名です。
organization	データセットの構造です。 – 0: 順次 – 1: 相対 – 2: 索引
access_mode	アクセス・モードです。 – 0: 順次 – 1: 任意

項目	説明
	– 2: 両方

データセットを開くとき、tcppfh_open() APIのパラメータでオープン・モードを指定する必要があります。以下は、サポートされるデータセットのオープン・モードです。

マクロ	オープン・モード	説明
TCFH_OPEN_INPUT	0	INPUTモードです。
TCFH_OPEN_OUTPUT	1	OUTPUTモードです。
TCFH_OPEN_INOUT	2	INPUT/OUTPUTモードです。
TCFH_OPEN_EXTEND	3	EXTENDモードです。

データセットが正常に開かれた場合、ファイル構造体のfile_status、file_path、file_handleなどの値が設定されます。

B.2.2. tcfh_close()

レコード処理のために開いていたデータセットを閉じます。データセットを閉じる操作は、アプリケーションでデータセットを読み書きするなどのI/Oを行った後、アプリケーションと物理的なデータセットとの接続を切断する作業です。

- プロトタイプ

```
int tcfh_close(tcfh_file_t *file, int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	閉じるデータセットの情報を保存したファイル構造体です。
flags(入力)	データセットのクローズ・オプションです。 – WITH LOCK

- 戻り値

正常に実行された場合は0が返され、エラーが生じた場合は負数のエラーコードが返されます。

- 参考

以下は、CLOSEコマンドを使用時の注意事項です。

- tcfh_close() APIのfileパラメータには、tcfh_open() APIに提供されたファイル構造体のポインタを渡します。

- tcfh_close() APIを使用して正常に閉じたデータセットは、tcfh_open() APIを使用して再オープンすることができます。

B.3. レコード・アクセス

B.3.1. tcfh_start()

相対データセット、または索引データセットで次に読み取るレコードの位置に内部レコード・ポインターを移動させます。移動させるレコードの位置は、相対データセットの場合は、ファイル構造体のrelative_keyフィールドに相対レコード番号を指定し、索引データセットの場合は、tcfh_start() APIのkeyパラメータにキー値を指定します。

- プロトタイプ

```
int tcfh_start(tcfh_file_t *file, char *key, int keylen, int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
key(入出力)	索引データセットのキー値です。
keylen(入力)	索引データセットのキー長です。
flags(入力)	データセットのSTARTオプションです。 - EQUAL - GTEQ

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 注意事項

以下は、STARTコマンドを使用時の注意事項です。

- データセットをINPUTまたはINOUTモードで開いた場合にのみ、STARTコマンドを実行できます。
- コマンドを実行した後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。
- 相対データセットの場合、RRNはファイル構造体のrelative_keyフィールドを使用します。

B.3.2. tcfh_read()

データセットのレコードを順次に読み取るか、キーを指定してランダムに読み取ります。

ランダムにレコードを読み取る際、相対データセットの場合は、ファイル構造体のrelative_keyフィールドに相対レコード番号を指定し、索引データセットの場合は、tcfh_read() APIのkeyパラメータにキー値を指定します。

順次にデータセットのレコードを読み取る場合は、tcfh_read() APIのflagsパラメータにTCFH_READ_NEXTマクロを指定し、ランダムにレコードを読み取る場合は、TCFH_READ_DEFAULTマクロを指定します。

可変長レコード・データセットの場合は、READコマンドを使用してデータセットのレコードを読み取るとき、ファイル構造体のcur_reclenフィールドを調べて読み取ったレコードの長さを取得することができます。取得したレコード長は、tcfh_read() APIのbufパラメータに指定されたバッファーにコピーされたレコード・データの長さ、つまり、RDWを除いたレコードのデータ部分の長さです。

- プロトタイプ

```
tcfh_read(tcfh_file_t *file, char *key, int keylen, char *buf, int buflen,
          int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
key(入出力)	索引データセットのキー値です。
keylen(入力)	索引データセットのキー長です。
buf(出力)	レコード・データを保存するためのバッファーです。
buflen(入力)	レコード・データのバッファー・サイズです。
flags(入力)	データセットのREADオプションです。 – DEFAULT – NEXT

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 注意事項

以下は、READコマンドを使用時の注意事項です。

- データセットをINPUTまたはINOUTモードで開いた場合にのみ、READコマンドを実行できます。
- コマンドを実行した後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。

- 相対データセットの場合、RRNはファイル構造体のrelative_keyフィールドを使用します。
- 読み取ったレコードのデータ長は、ファイル構造体のcur_reclenフィールドで取得します。

B.3.3. tcfh_write()

データセットに新しいレコードを記録します。順次データセットはデータセットの最後にレコードを記録し、相対データセットはファイル構造体のrelative_keyフィールドに指定された相対レコード番号の位置に記録します。索引データセットは、tcfh_write() APIのkeyパラメータに指定されたキー順にレコードが保存されます。

可変長レコード・データセットの場合は、WRITEコマンドを使用してデータセットのレコードを記録するとき、記録するレコードの長さをtcfh_write() APIのbuflenパラメータに指定します。指定するレコードの長さは、tcfh_write() APIのbufパラメータで指定されたバッファに保存されているレコード・データの長さ、つまり、RDWを除いたレコードのデータ部分の長さを指定します。

● プロトタイプ

```
tcfh_write(tcfh_file_t *file, char *key, int keylen, char *buf, int buflen,
           int flags);
```

● パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
key(入出力)	索引データセットのキー値です。
keylen(入力)	索引データセットのキー長です。
buf(出力)	レコード・データを保存しているバッファです。
buflen(入力)	レコード・データの長さです。
flags(入力)	データセットのWRITEオプションです。

● 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

● 注意事項

以下は、WRITEコマンドを使用時の注意事項です。

- OUTPUT、INOUTまたはEXTENDモードで開いた場合にのみ、WRITEコマンドを使用できます。
- INOUTモードで開いた場合、順次アクセスによる書き込みはできません。
- コマンドを実行した後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。

- 相対データセットの場合、RRNはファイル構造体のrelative_keyフィールドを使用します。

B.3.4. tcfh_rewrite()

データセットにある既存のレコードの内容を変更します。相対データセットは、ファイル構造体のrelative_keyフィールドに指定された相対レコード番号の位置にあるレコードを変更します。索引データセットは、tcfh_rewrite() APIのkeyパラメータで指定されたキー順に合ったレコードが変更されます。

可変長レコード・データセットの場合は、REWRITEコマンドを使用してデータセットのレコードを変更するとき、変更するレコードの長さをtcfh_rewrite() APIのbuflenパラメータに指定します。指定するレコードの長さは、tcfh_rewrite() APIのbufパラメータに指定されたバッファに保存されているレコード・データの長さ、つまり、RDWを除いたレコードのデータ部分の長さを指定します。

- プロトタイプ:

```
tcfh_rewrite(tcfh_file_t *file, char *key, int keylen, char *buf, int buflen,
            int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
key(入力)	索引データセットのキー値です。
keylen(入力)	索引データセットのキー長です。
buf(入力)	レコード・データを保存しているバッファです。
buflen(入力)	レコード・データの長さです。
flags(入力)	データセットのREWRITEオプションです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 注意事項

以下は、REWRITEコマンドを使用時の注意事項です。

- データセットをINOUTモードで開いた場合にのみ、REWRITEコマンドを使用できます。
- 順次アクセスの場合、REWRITEコマンドの実行前に、READコマンドで変更するレコードを読み取ります。
- コマンドを実行した後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。
- 相対データセットの場合、RRNはファイル構造体のrelative_keyフィールドを使用します。

B.3.5. tcfh_delete()

相対データセットまたは索引データセットのレコードを削除します。相対データセットの場合は、ファイル構造体のrelative_keyフィールドに指定された相対レコード番号の位置にあるレコードを削除します。索引データセットの場合は、tcfh_delete() APIのkeyパラメータで指定されたキー順に合ったレコードが削除されます。

- プロトタイプ

```
int tcfh_delete(tcfh_file_t *file, char *key, int keylen, int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
key(入力)	索引データセットのキー値です。
keylen(入力)	索引データセットのキー長です。
flags(入力)	データセットのDELETEオプションです。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 注意事項

以下は、DELETEコマンドを使用時の注意事項です。

- データセットをINOUTモードで開いた場合にのみ、DELETEコマンドを使用できます。
- 順次アクセスの場合、DELETEコマンドの実行前に、READコマンドで削除するレコードを読み取る必要があります。
- コマンドを実行した後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。
- 相対データセットの場合、RRNはファイル構造体のrelative_keyフィールドを使用します。

B.4. PDSメンバーの処理

B.4.1. tcfh_bldl()

PDSのディレクトリー一覧を構成します。

- プロトタイプ

```
int tcfh_bldl(tcfh_file_t *file, char *filter);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
filter(入力)	リストを構成するメンバーに制限を与える値です。データセットのメンバー全体をリストで構成する場合は、アスタリスク(*)またはNULL値を指定します。「ABC」で始まるメンバーのみを構成する場合は、filterに「ABC」値を指定します。

- 戻り値

正常に実行された場合はディレクトリー一覧を構成したメンバーの数を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

以下は、BLDLコマンドを使用時の注意事項です。

- コマンドの実行後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。

B.4.2. tcfh_bldl_info()

PDSのディレクトリー一覧を検索します。tcfh_bldl()を実行した後に使用できます。

データセットのレコードを順次に読み取る場合は、tcfh_read()のflagsパラメータにTCFH_READ_NEXTマクロを指定し、ランダムに読み取る場合は、TCFH_READ_DEFAULTマクロを指定します。

可変長レコード・データセットの場合は、READコマンドを使用してデータセットのレコードを読み取るとき、ファイル構造体のcur_reclenフィールドを調べて読み取ったレコードの長さを取得できます。取得したレコード長は、tcfh_read()のbufパラメータで指定されたバッファーにコピーされたレコード・データの長さ、つまり、RDWを除いたレコードのデータ部分の長さです。

- プロトタイプ

```
int tcfh_bldl_info(tcfh_file_t *file, int number, tcfh_membstat_t *stat);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
number(入力)	ディレクトリー一覧での相対アドレス値です。
stat(出力)	索引データセットのキー長です。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

メンバー情報を検索するには、メンバーの情報を保存したファイル構造体のポインターを提供する必要があります。このファイル構造体は、tcfh.hヘッダー・ファイルに定義されています。

```
/* ----- file I/O block ----- */

typedef struct tcfh_membstat_s {
    char        member[64];
    int64_t     size;
    char        lastmoddt[8+2]; /* last modified date */
    char        lastmodtm[6+2]; /* last modified time */
} tcfh_membstat_t;
```

上記のファイル構造体で情報を提供する項目は以下のとおりです。

項目	説明
member(出力)	PDSのメンバー名が記述されます
size(出力)	メンバーが保存されたUNIXファイルのサイズを表示します。
lastmoddt(出力)	メンバーの最終修正日付を表示します。
lastmodtm(出力)	メンバーの最終修正時刻を表示します。

B.4.3. tcfh_find_by_bldl()

PDSのメンバーにアクセスします。tcfh_bldl()を実行した後に使用できます。

- プロトタイプ

```
int tcfh_find_by_bldl(tcfh_file_t *file, int num);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
num(入力)	索引データセットのキー値です。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

以下は、FINDコマンドを使用時の注意事項です。

- コマンドの実行後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。

B.4.4. tcfh_find_by_name()

PDSのメンバーにアクセスします。tcfh_bldl()の実行に関係なく使用できます。

- プロトタイプ

```
int tcfh_find_by_name(tcfh_file_t *file, char *name);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
name(入力)	アクセスするメンバー名を指定します。

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 参考

以下は、FINDコマンドを使用時の注意事項です。

- コマンドの実行後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。

B.4.5. tcfh_stow()

PDSにメンバーを追加、取り替え、削除、初期化、およびメンバー名を変更します。

- プロトタイプ

```
int tcfh_stow(tcfh_file_t *file, char *name, int flags);
```

- パラメータ

パラメータ	説明
file(入出力)	処理するデータセットの情報を保存したファイル構造体です。
name(入力)	処理するメンバー名を記述します。スラッシュ(/)記号を使って変更前と変更後のメンバー名を区切ります。 (例: "BEFORE/AFTER")
flags(入力)	データセットのSTOWオプションです。

パラメータ	説明
	– TCFH_STOW_ADD – TCFH_STOW_DELETE – TCFH_STOW_CHANGE – TCFH_STOW_REPLACE – TCFH_STOW_INITIALIZE

- 戻り値

正常に実行された場合は0を返し、エラーが生じた場合は負数のエラーコードを返します。

- 注意事項

以下は、STOWコマンドを使用時の注意事項です。

- メンバーを追加、取り替える場合、WRITEコマンドが先に実行される必要があります。
- コマンドの実行後、ファイル構造体のfile_statusでファイルの状態をチェックすることができます。

B.5. ファイル・ステータス・コード

データセットの読み書きを実行した後、データセットの状態を確認するには、データセットの情報を保存しているファイル構造体のfile_statusフィールドを調べます。file_statusフィールドには、データセットの状態コードを保存されています。

以下は、データセットのステータス・コードについての説明です。

コード	説明
00	作業が正常に完了した場合です。
02	索引ファイルの場合、重複キーが許可されているデータセットに重複キーが発生した場合です。
10	次のレコードがない場合です。（ファイルを最後まで読み取った場合）
22	索引ファイルや相対ファイルに対し、重複キー・エラーが発生した場合です。
23	キーに該当するレコードが見つからなかった場合です。
37	指定されたオープン・モードがサポートされない場合です。
38	ロックされたファイルを開こうとした場合です。
39	データセットの属性が正しく指定されていない場合です。
41	すでに開いているファイルを開こうとした場合です。

コード	説明
42	すでに閉じられているファイルを閉じようとした場合です。
43	順次モードでREWRITEまたはDELETEコマンドの実行前のコマンドがREADコマンドでない場合です。
44	INPUT/INOUT以外のモードで、READ/STARTコマンドを実行した場合です。
47	INPUT/INOUT以外のモードで、READ/STARTコマンドを実行した場合です。
48	OUTPUT/INOUT/EXTEND以外のモードで、WRITEコマンドを実行した場合です。
49	INOUT以外のモードで、DELETE/REWRITEコマンドを実行した場合です。
99	システム・エラーが発生した場合です。

付録 C. DCBハンドリングAPI

本付録では、DCBハンドリングAPIを使用して、JCLで割り当てられたDDの属性情報を検索したり、一部の属性を変更したりする方法について説明します。

C.1. 概要

OpenFrameが提供するDCB(Data Control Block)Handling APIを使用すると、JCLで割り当てられたDD(データ定義)の属性情報を検索したり、一部の属性を変更したりすることができます。また、アプリケーション・プログラムで動的割り当てを実行することも可能です。

DCB処理APIを使用してプログラムを作成するには、OpenFrameバイナリと一緒に配布されるtdcb.hヘッダー・ファイルをユーザー・プログラムに含める必要があります、ユーザー・プログラムをコンパイルする際、libtdcb.soライブラリをリンクする必要があります。

DCBハンドリングAPIは、以下のように分類できます。

● 割り振りハンドル (Allocation Handle)

API	説明
tdcb_get_handle()	JCLのDD文で割り当てられたDCB情報をDD名で検索します。
tdcb_discard_handle()	tdcb_get_handle() APIを実行して検索された割り当て情報を解除します。
tdcb_print_info()	tdcb_get_handle() APIを実行して検索された割り当て情報を出力します。

● DCB情報

API	説明
tdcb_get_type()	DCBタイプと順次連結(Concatenation)の数を取得します。
tdcb_maxlrecl()	論理レコードの最大長と平均長を取得します。
tdcb_nvsm_info()	非VSAMデータセットのDSORG属性とRECFM属性を取得します。

● 属性の取得 (Get Attributes)

API	説明
tdcb_get_dsname()	データセット名とメンバー名を取得します。
tdcb_get_volser()	データセットのボリューム・シリアル情報を取得します。

API	説明
tdcb_get_flags()	データセットのその他の属性を取得します。
tdcb_get_blksize()	データセットのブロック・サイズとレコード長を取得します。
tdcb_get_key_info()	データセットのキーの位置とキー長の情報を取得します。
tdcb_get_vsamtype()	VSAMデータセットの種類とAIX属性を取得します。

● 属性の設定 (Set Attributes)

API	説明
tdcb_set_lrecl()	データセットの論理レコード長を指定します。
tdcb_set_recfm()	データセットのRECFM属性を指定します。
tdcb_set_blksize()	データセットのブロック・サイズを指定します。
tdcb_set_keylen()	データセットのキー長とキーの位置情報を指定します。

● 動的割り当て (Dynamic Allocation)

API	説明
tdcb_allocate()	データセットに対して動的割り当てを実行します。
tdcb_concatenate()	動的割り当てを実行したDDにデータセットの連結を行います。
tdcb_unallocate()	動的割り当てを実行したDDの割り当てを解除します。

C.2. 割り振りハンドル

C.2.1. [tdcb_get_handle\(\)](#)

JCLのDD文で割り当てられたDCB情報をDD名で検索します。 [tdcb_get_handle\(\)](#) APIを実行すると、JCLによって共有メモリに構成されたDCB情報の一部をローカル・メモリ領域に再構成し、その後の割り当て情報へのアクセスを容易にします。

正常に割り当て情報を検索した場合は、負数以外の割り振りハンドルを返します。返された割り振りハンドルは、以降のDCB情報検索関数や属性を変更する関数などで使用されます。

● プロトタイプ

```
int tdcb_get_handle(char *ddname);
```


- パラメータ

パラメータ	説明
ddname(入力)	取得する割り当て情報のDD名です。

- 戻り値

正常に割り当て情報を検索した場合は、負数以外の割り振りハンドルを返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.2.2. tdcb_discard_handle()

tdcb_get_handle() APIを実行して検索された割り当て情報を解除します。つまり、ローカル・メモリ領域に構成されたDCB情報が解除されます。したがって、同じDDのDCB情報を再使用するには、tdcb_get_handle() APIを再び呼び出す必要があります。

tdcb_get_handle() APIを呼び出して取得した割り当て情報を、tdcb_discard_handle() APIを使用して必ず解除する必要はありませんが、同じDDに対してtdcb_get_handle() APIを繰り返し呼び出すと、そのDD情報がすでにローカル・メモリ領域に構成されているという警告メッセージが発生します。

Cモジュールでtdcb_discard_handle() APIを使用して割り当て情報を解除していないDDに対し、COBOLモジュールでOPEN文が実行されても、内部的にはtdcb_get_handle() APIと類似したAPIが呼び出されるため、同じ警告メッセージが発生します。

- プロトタイプ

```
int tdcb_discard_handle(int handle);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.2.3. tdcb_print_info()

tdcb_get_handle() APIを実行して検索された割り当て情報を出力します。

以下は、出力される割り当て情報に含まれる内容です。

```
DD名, ジョブ名, 割り当て情報, 割り当て日, 割り当て時間, 順次連結数  
+ [順番] スロット番号, DCBタイプ, DCB名, DISPOSITION
```

- プロトタイプ

```
int tdcdb_print_info(int handle);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcdb_get_handle() APIを実行して取得した割り振りハンドルです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.3. DCB情報

C.3.1. tdcdb_get_type()

tdcdb_get_handle() APIを実行して検索された割り当て情報からDCBのタイプと順次連結数を取得します。

DCBタイプのtdcdb.hヘッダー・ファイルには、以下のようなマクロが定義されています。

```
#define TDCDB_DCB_TYPE_NONE          0x00    /* Unknown */
#define TDCDB_DCB_TYPE_NVSM         0x01    /* Non-VSAM */
#define TDCDB_DCB_TYPE_TSAM         0x02    /* VSAM */
#define TDCDB_DCB_TYPE_ISAM         0x03    /* ISAM */
```

OpenFrameシステムでは、2つ以上の非VSAMデータセットを連結して、1つのデータセットであるかのように連続して検索できる方法を提供しています。これを順次連結(Sequential Concatenation)といいます。順次連結の数は、連結するデータセットの数を意味します。

OpenFrameシステムでは、VSAMデータセットの順次連結はサポートしていません。DCBタイプがTSAMである場合、countパラメータには順次連結の数ではなく、ベース・クラスターとそれに連結されている代替索引の総数が含まれます。

- プロトタイプ

```
int tdcdb_get_type(int handle, int *dcb_type, int *count);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcdb_get_handle() APIを実行して取得した割り振りハンドルです。
dcb_type(出力)	与えられた割り当て情報のDCBタイプを保存するバッファです。

パラメータ	説明
count(出力)	順次連結の数、またはベース・クラスターと代替索引の総数を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.3.2. tdc_b_maxlrecl()

tdcb_get_handle() APIを実行して検索された割り当て情報から、論理レコードの最大長と平均長を取得します。

順次連結されていない非VSAMデータセットや固定長VSAMデータセットの場合、最大レコード長と平均レコード長が同じです。一方、順次連結されている非VSAMデータセットや可変長VSAMデータセットの場合は、最大レコード長と平均レコード長が異なる場合があります。

可変長非VSAMデータセットのレコード長は、JCLでデータセットを割り当てるときに指定されたLRECL値が渡されます。つまり、RDWを含む論理レコード長が取得されます。

- プロトタイプ

```
int tdc_b_maxlrecl(int handle, int *maxlrecl, int *avglrecl);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
maxlrecl(出力)	与えられた割り当て情報の論理レコードの最大長を保存するバッファです。
avglrecl(出力)	与えられた割り当て情報の論理レコードの平均長さを保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.3.3. tdc_b_nvsm_info()

tdcb_get_handle() APIを実行して検索された割り当て情報から、非VSAMデータセットのDSORG属性とRECFM属性を取得します。

非VSAMデータセットのDSORG属性は、データセットの構造を指定します。以下のようなデータセット構造が表現できます。

DSORG	説明
PS(Physical Sequential)	SAMデータセットです。
IS(Indexed Sequential)	ISAMデータセットです。
PO(Partitioned)	PDSまたはPDSEです。
DA(Direct)	DAMデータセットです。

非VSAMデータセットのRECFM属性は、レコードの特徴を指定します。RECFM属性を指定する文字は、それぞれ以下のような意味を持ちます。

RECFM	説明
F	固定長レコードです。
V	可変長レコードです。
L	行順次レコードです。
U	不定長レコード形式です。
B	ブロック単位のI/Oをサポートします。
A	ANSI制御文字を含みます。

固定長レコードに対してブロック単位のI/OをサポートするデータセットのRECFM属性は「RECFM=FB」形式で指定します。また、可変長レコードにANSI制御文字が含まれているデータセットのRECFM属性は、「RECFM=VA」形式で指定します。

順次連結されている非VSAMデータセットの場合、最初のデータセットのDSORG属性とRECFM属性が返されます。（原則として、メインフレームでは基本属性が異なる非VSAMデータセットの順次連結は禁じられています。）

VSAMデータセットが固定長か可変長かによって、DSORG=PS、RECFM=FBまたはDSORG=PS、RECFM=VBの値が返されます。

割り当て情報からDSORG属性を取得するには、最小8バイト以上の文字列バッファ・ポインターをdsorgパラメータに指定する必要があります。また、RECFM属性を取得するには、最小8バイト以上の文字列バッファ・ポインターをrecfmパラメータに指定する必要があります。ユーザー文字列バッファが指定された場合は、DSORG属性とRECFM属性がNULL終了文字と一緒にユーザー・バッファにコピーされます。

- プロトタイプ

```
int tdcdb_nvsm_info(int handle, char *dsorg, char *recfm);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
dsorg(出力)	与えられた割り当て情報のDSORG属性を保存するバッファです。
recfm(出力)	与えられた割り当て情報のRECFM属性を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.4. 属性の取得

C.4.1. tdcb_get_dsname()

tdcb_get_handle() APIを実行して検索された割り当て情報から、特定のDCBコンポーネントのデータセット名とメンバー名を取得します。

DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットか、あるいはVSAM割り当て情報におけるベース・クラスターまたは代替索引のいずれかを意味します。

割り当てられたデータセット名を取得するには、最小45バイト以上の文字列バッファ・ポインターを、dsnameパラメータに指定する必要があります。（NULL終了文字と一緒にユーザー・バッファにコピーされます。）

割り当てられたデータセットがPDSまたはPDSEデータセットのメンバーである場合は、メンバー名を取得することができます。最小9バイト以上の文字列バッファ・ポインターをmemberパラメータに指定する必要があります。（NULL終了文字と一緒にユーザー・バッファにコピーされます。）

- プロトタイプ

```
int tdcb_get_dsname(int handle, int index, char *dsname, char *member);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結の順番、あるいはベース・クラスターまたは代替索引の指定番号です。
dsname(出力)	DCBコンポーネントのデータセット名を保存するバッファです。
member(出力)	DCBコンポーネントのメンバー名を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.4.2. tdc_b_get_volser()

tdcb_get_handle() APIを実行して検索された割り当て情報から、特定のDCBコンポーネントのボリューム・シリアル情報を取得します。DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットか、あるいはVSAM割り当て情報におけるベース・クラスターまたは代替索引のいずれかを意味します。

割り当てられたDCBコンポーネントのボリューム・シリアル情報を取得するには、最小9バイト以上の文字列バッファ・ポインタをvolserパラメータに指定する必要があります。（NULL終了文字と一緒にユーザー・バッファにコピーされます）

- プロトタイプ

```
int tdc_b_get_volser(int handle, int index, char *volser);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結の順番、あるいはベース・クラスターまたは代替索引の指定番号です。
volser(出力)	DCBコンポーネントのボリューム・シリアル情報を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.4.3. tdc_b_get_flags()

tdcb_get_handle() APIを実行して検索された割り当て情報から、特定のDCBコンポーネントのその他の属性を取得します。

DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットか、あるいはVSAM割り当て情報におけるベース・クラスターまたは代替索引のいずれかを意味します。

DCBコンポーネントのその他の属性とは、tdcb.hヘッダー・ファイルに定義されている以下のようなマクロを意味します。

```
#define TDCB_LRECL_NOT_SET      0x01
#define TDCB_RECFM_NOT_SET      0x04
#define TDCB_BLKSIZE_NOT_SET    0x08
#define TDCB_KEYLEN_NOT_SET     0x40
#define TDCB_IS_TAPE_DATASET    0x10
#define TDCB_IS_OPEN_INDICATOR 0x20
```

以下は、それぞれのマクロについての説明です。

マクロ	説明
TDCB_LRECL_NOT_SET	カタログやJCLでデータセットの論理レコード長が指定されていない場合です。
TDCB_RECFM_NOT_SET	カタログやJCLでデータセットのRECFM属性が指定されていない場合です。
TDCB_BLKSIZE_NOT_SET	カタログやJCLでデータセットのブロック・サイズが指定されていない場合です。
TDCB_KEYLEN_NOT_SET	カタログやJCLでデータセットのキー長が指定されていない場合です。
TDCB_IS_TAPE_DATASET	テープ・ボリューム・デバイスに割り当てられたデータセットです。
TDCB_IS_OPEN_INDICATOR	アプリケーションからまだOPENAPIが呼び出されていない場合です。

● プロトタイプ

```
tdcb_get_flags(int handle, int index, int *flags);
```

● パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結の順番、あるいはベース・クラスターまたは代替索引の指定番号です。
flags(出力)	DCBコンポーネントのその他の属性情報を保存するバッファです。

● 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.4.4. tdc_b_get_blksize()

tdcb_get_handle() APIを実行して検索された割り当て情報から、特定のDCBコンポーネントのブロック・サイズとレコード長を取得します。

DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットか、あるいはVSAM割り当て情報におけるベース・クラスターまたは代替索引のいずれかを意味します。

可変長非VSAMデータセットのレコード長は、JCLで該当するデータセットを割り当てるときに指定されたLRECL値が渡されます。つまり、RDWを含む論理レコード長が取得されます。

割り当てられたDCBコンポーネントのレコード長の情報が不要な場合は、reclenパラメータにNULLを指定します。

- プロトタイプ

```
int tdc_b_get_blksize(int handle, int index, int *blksize, int *reclen);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結の順番、あるいはベース・クラスターまたは代替索引の指定番号です。
blksize(出力)	DCBコンポーネントのブロック・サイズを保存するバッファです。
reclen(出力)	DCBコンポーネントのレコード長を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.4.5. tdc_b_get_key_info()

tdcb_get_handle() APIを実行して検索された割り当て情報から、特定のDCBコンポーネントのキーの位置とキー長の情報を取得します。

DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットか、あるいはVSAM割り当て情報におけるベース・クラスターまたは代替索引のいずれかを意味します。

割り当てられたDCBコンポーネントのキーの情報がない場合（割り当てられたデータセットが非VSAMデータセットであり、DSORGがPSかPOである場合）、キーの位置とキーの長さ情報がすべて0で返されます。

- プロトタイプ


```
int tdcdb_get_key_info(int handle, int index, int *keypos, int *keylen);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcdb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結の順番、あるいはベース・クラスターまたは代替索引の指定番号です。
keypos(出力)	DCBコンポーネントのキーの位置を保存するバッファです。
keylen(出力)	DCBコンポーネントのキー長を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.4.6. tdcdb_get_vsamtype()

tdcdb_get_handle() APIを実行して検索された割り当て情報から、VSAM DCBコンポーネントのVSAMデータセットの種類とAIX属性を取得します。

DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットか、あるいはVSAM割り当て情報におけるベース・クラスターまたは代替索引のいずれかを意味します。

VSAMデータセットには、以下のようなマクロがtdcdb.hヘッダー・ファイルに定義されています。

```
#define TDCB_VSAM_TYPE_ESDS      0x01    /* ESDS */
#define TDCB_VSAM_TYPE_KSDS     0x02    /* KSDS */
#define TDCB_VSAM_TYPE_RRDS     0x04    /* RRDS */
#define TDCB_VSAM_TYPE_VRDS     0x05    /* VRDS */
```

AIX属性とは、tdcdb.hヘッダー・ファイルに定義されている以下のようなマクロです。

```
#define TDCB_RGATTR_EAIX        0x01
#define TDCB_RGATTR_KAIX        0x02
#define TDCB_RGATTR_UPGRADE     0x04
```

以下は、各マクロの説明です。

マクロ	説明
TDCB_RGATTR_EAIX	VSAM DCBコンポーネントはESDSの代替索引です。
TDCB_RGATTR_KAIX	VSAM DCBコンポーネントはKSDSの代替索引です。
TDCB_RGATTR_UPGRADE	代替索引コンポーネントは、UPGRADE属性を持ちます。

- プロトタイプ

```
int tdc_b_get_vsamtype(int handle, int index, int *vsamtype, int *rgattr);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結の順番、あるいはベース・クラスターまたは代替索引の指定番号です。
vsamtype(出力)	VSAM DCBコンポーネントのVSAMデータセットの種類を保存するバッファです。
rgattr(出力)	VSAM DCBコンポーネントのAIX属性を保存するバッファです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.5. 属性の設定

C.5.1. tdc_b_set_lrecl()

tdcb_get_handle() APIを実行して検索された割り当て情報で、特定のDCBコンポーネントの論理レコード長を指定します。DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットを意味します。

tdcb_set_lrecl() APIを使用して論理レコード長を指定する前に、tdcb_get_flags() APIを使用して、該当するDCBコンポーネントの論理レコード長がカタログやJCLで指定されていないことを確認してください。

可変長非VSAMデータセットのレコード長は、JCLでデータセットを割り当てるときと同様に、RDWを含む論理レコード長を指定します。

- プロトタイプ

```
int tdc_b_set_lrecl(int handle, int index, int lrecl);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結されている非VSAMデータセットの順番または0です。

パラメータ	説明
lrecl(入力)	DCBコンポーネントに指定する論理レコード長です。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.5.2. tdc_b_set_recfm()

tdcb_get_handle() APIを実行して検索された割り当て情報で、特定のDCBコンポーネントのRECFM属性を指定します。DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットを意味します。

tdcb_set_recfm() APIを使用してRECFM属性を指定する前に、tdcb_get_flags() APIを使用して、該当するDCBコンポーネントのRECFM属性がカタログやJCLで指定されていないことを確認してください。

- プロトタイプ

```
int tdc_b_set_recfm(int handle, int index, char *recfm);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結されている非VSAMデータセットの順番または0です。
recfm(入力)	DCBコンポーネントに指定するRECFM属性です。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.5.3. tdc_b_set_blksize()

tdcb_get_handle() APIを実行して検索された割り当て情報で、特定のDCBコンポーネントのブロック・サイズを指定します。DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットを意味します。

tdcb_set_blksize() APIを使用してブロック・サイズを指定する前に、tdcb_get_flags() APIを使用して、該当するDCBコンポーネントのブロック・サイズがカタログやJCLで指定されていないことを確認してください。

- プロトタイプ

```
int tdc_b_set_blksize(int handle, int index, int blksize);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結されている非VSAMデータセットの順番または0です。
blksize(入力)	DCBコンポーネントに指定するブロック・サイズです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.5.4. tdc_b_set_keylen()

tdcb_get_handle() APIを実行して検索された割り当て情報で、特定のDCBコンポーネントのキー長とキーの位置情報を指定します。DCBコンポーネントとは、順次連結されている非VSAM割り当て情報における特定の1つのデータセットを意味します。

tdcb_set_keylen() APIでキー長とキーの位置情報を指定する前に、tdcb_get_flags() APIを使用して、該当するDCBコンポーネントのキー長がカタログやJCLで指定されていないことを確認してください。

- プロトタイプ

```
int tdc_b_set_keylen(int handle, int index, int keylen, int keyoff);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_get_handle() APIを実行して取得した割り振りハンドルです。
index(入力)	順次連結されている非VSAMデータセットの順番または0です。
keylen(入力)	DCBコンポーネントに指定するキーの長さです。
keyoff(入力)	DCBコンポーネントに指定するキーの位置です。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーコード、エラーメッセージと一緒に-1を返します。

C.6. 動的割り当て

C.6.1. tdcb_allocate()

JCLに技術していないデータセットに対し、アプリケーション・プログラムで割り当てを実行してデータセットにアクセスする場合に使用します。

tdcb_allocateを使用して動的割り当てを実行した後、データセットのオープン、およびその他のI/Oを実行することができます。

● プロトタイプ

```
int tdcb_allocate(char *ddname, char *ddparam, int flags);
```

● パラメータ

パラメータ	説明
ddname(入力)	DD名を指定します。
ddparam(入力)	DD属性を文字列で指定します。 各属性はコンマ(,)で区切ります。 – DSNNAME : データセット名を記述します。(例: DSNNAME=ABC) – DISP : 使用可能ステータス、正常終了DISP、異常終了DISPを指定します。(例: DISP=(NEW、CATLG)) – DATACLAS : データセットのDATACLASを指定します。(例: DATACLAS=ACLS) – MGMTCLAS : データセットのMGMTCLASを指定します。 – STORCLAS : データセットのSTORCLASを指定します。 – VOL=SER : ボリューム・シリアル情報を指定します。(例: VOL=SER=DEFVOL) – SPACE : SPACE属性を指定します。(例: SPACE=(CYL,(1,1),RLSE)) – DSORG : DSORG属性を指定します。(例: DSORG=PS) – RECFM : RECFM属性を指定します。 – BLKSIZE : BLKSIZE属性を指定します。(例: BLKSIZE=4096) – LRECL : LRECL属性を指定します。 – KEYLEN : KEYLEN属性を指定します。 – KEYOFF : KEYOFF属性を指定します。 – DCB : DCB属性を指定します。(例: DCB=(RECFM=FB、LRECL=80))

パラメータ	説明
flags(入力)	割り当てる際のオプションを指定します。 – TDCB_ALLOCATE_DEFAULT : デフォルトの指定オプションです。 – TDCB_ALLOCATE_NOLOCK : ロックを無効にするときに使用します。 – TDCB_ALLOCATE_NOACHK : 権限チェックを実行しません。 – TDCB_ALLOCATE_TEMPORARY : 一時データセットを指定します。 – TDCB_ALLOCATE_DISPACHK : 割り当ての前にディスポジションへの権限チェックを行います。 – TDCB_ALLOCATE_UNNAMED_TAPE : 名前を指定しないテープ・データセットを指定します。

- 戻り値

正常に割り当てを実行した場合は、負数以外の割り振りハンドルを返し、エラーが発生した場合は、エラーが発生した関数をエラーメッセージとして出力し、負数のエラーコードを返します。エラーコードは、oferrorツールで確認できます。

C.6.2. tdcb_concatenate()

tdcb_allocate() APIから返されたハンドルの値をパラメータとして、データセットの連結を行います。

- プロトタイプ

```
int tdcb_concatenate(int handle, char *ddparam, int flags);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_allocate() APIから返されたハンドルを指定します。
ddparam(入力)	DD属性を文字列で指定します。 各属性はコンマ(,)で区切ります。 – DSNAME : データセット名を記述します。(例: DSNAME=ABC) – DISP : 使用可能ステータス、正常終了DISP、異常終了DISPを指定します。(例: DISP=(NEW、CATLG)) – DATACLAS : データセットのDATACLASを指定します。(例: DATACLAS=ACLS)

パラメータ	説明
	– MGMTCLAS : データセットのMGMTCLASを指定します。 – STORCLAS : データセットのSTORCLASを指定します。 – VOL=SER : ボリューム・シリアル情報を指定します。(例: VOL=SER=DEFVOL) – SPACE : SPACE属性を指定します。(例: SPACE=(CYL,(1,1),RLSE)) – DSORG : DSORG属性を指定します。(例: DSORG=PS) – RECFM : RECFM属性を指定します。 – BLKSIZE : BLKSIZE属性を指定します。(例: BLKSIZE=4096) – LRECL : LRECL属性を指定します。 – KEYLEN : KEYLEN属性を指定します。 – KEYOFF : KEYOFF属性を指定します。 – DCB : DCB属性を指定します。(例: DCB=(RECFM=FB、LRECL=80))
flags(入力)	連結時のオプションを指定します。 – TDCB_CONCATENATE_DEFAULT : デフォルトの指定オプションです。 – TDCB_CONCATENATE_NOLOCK : ロックを無効にするときに使用します。 – TDCB_CONCATENATE_SPOOL : スプール属性を持つときに指定します。 – TDCB_CONCATENATE_NOCHK : 権限チェックを実行しません。 – TDCB_CONCATENATE_TEMPORARY : 一時データセットを指定します。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーが発生した関数をエラーメッセージとして出力し、負数のエラーコードを返します。エラーコードは、oferrorツールで確認できます。

C.6.3. tdcb_unallocate()

tdcb_allocate() APIを実行して動的割り当てを行ったデータセットを解除します。

データセットに対してCLOSEを実行した後、割り当ての解除を行います。

- プロトタイプ

```
int tdc_b_unallocate(int handle, int disp, int flags);
```

- パラメータ

パラメータ	説明
handle(入力)	tdcb_allocate() APIから返された割り振りハンドルです。
disp(入力)	正常終了か、あるいは異常終了かを指定します。 – TDCB_UNALLOCATE_DISP_NORMAL : 正常終了のときに指定します。 – TDCB_UNALLOCATE_DISP_ABNORMAL : 異常終了のときに指定します。
flags(入力)	割り当て解除のオプションを指定します。 – TDCB_UNALLOCATE_DEFAULT : デフォルトの指定オプションです。

- 戻り値

正常に実行された場合は0を返し、エラーが発生した場合は、エラーが発生した関数をエラーメッセージとして出力し、負数のエラーコードを返します。エラーコードは、oferrorツールで確認できます。

付録 D. FCDハンドリングAPI

本付録では、FCDハンドリングAPIを使用して、メインCOBOLアプリケーション・プログラムでオープンされたデータセットを、C言語で作成されたサブプログラムで読み書きする方法について説明します。

D.1. 概要

OpenFrameと一緒に提供されるFCD(File Control Description)ハンドリングAPIを使用して、メインCOBOLアプリケーション・プログラムでオープンされたデータセットを、C言語で作成されたサブプログラムで読み書きするなどのI/Oが実行できます。ただし、メインCOBOLアプリケーション・プログラムをMF-COBOLコンパイラでコンパイルする必要があります。

FCDハンドリングAPIライブラリは、データセットの情報を保存している構造体としてFCDを使用します。FCDは、MF-COBOLコンパイラによってコンパイルされた実行バイナリが実行される際、データセットを開くときに生成されるメモリ構造体です。したがって、本書で説明するTmax FCDハンドリングAPIライブラリは、MF-COBOLコンパイラによってコンパイルされた実行バイナリでのみ一緒に使用することができます。

参考

最初からCで作成されたアプリケーション・プログラムでデータセットのI/Oを行う場合は、FCDハンドリングAPIライブラリではなく、C File Handlerライブラリを使用することをお勧めします。

FCDハンドリングAPIを使用してプログラムを作成するには、OpenFrameバイナリと一緒に配布されるtfcd.hヘッダー・ファイルを、Cで作成されたサブプログラムに含める必要があります、そのサブプログラムをコンパイルする際、libtfcd.soライブラリをリンクする必要があります。

Tmax FCDハンドリングAPIは、以下のように分類できます。

● ファイル・オープン・セッション (File Open Session)

API	説明
tfcd_open()	FCD構造体に記述されているデータセットを開きます。
tfcd_close()	FCD構造体に記述されているデータセットを閉じます。

- レコード・アクセス (Record Access)

API	説明
tfcd_start()	相対データセットまたは索引データセットで次に読み取るレコードの位置に移動させます。
tfcd_read()	データセットのレコードを順次に読み取るか、またはキーを指定してランダムに読み取ります。
tfcd_write()	データセットに新しいレコードを書き込む(WRITE)か、または既存のレコードを再書き込み(REWRITE)します。
tfcd_writea()	レコードの先頭にASA制御文字を指定してデータセットに新しいレコードを書き込みます。
tfcd_delete()	データセットにある既存のレコードを削除します。 (RELATIVE/INDEXED)

- 特殊な操作 (Special Operations)

API	説明
tfcd_is_file_open()	FCD構造体に記述されているデータセットが開かれているかを確認します。
tfcd_get_status()	データセットにI/Oを実行した後、エラーの発生有無を確認するために状態コードを取得します。
tfcd_get_file_name()	FCD構造体に記述されているデータセットのDD名を取得します。
tfcd_get_rec_len()	FCD構造体に記述されているデータセットのレコード長を取得します。
tfcd_set_file_name()	FCD構造体に記述されているデータセットのDD名を指定します。
tfcd_set_relative_key()	FCD構造体に記述されているデータセットの相対キーを指定します。 (RELATIVE)
tfcd_set_key_id()	FCD構造体に記述されているデータセットのKEY IDを指定します。 (INDEXED)
tfcd_set_rec_len()	FCD構造体に記述されているデータセットのレコード長を指定します。

D.2. ファイル・オープン・セッション

D.2.1. tfcd_open()

FCD構造体に記述されているデータセットを開きます。データセットのオープンは、JCLでDD文によって割り当てられたデータセットに対し、実際にレコードを読み書きするなどのI/Oを行うための準備手順です。

データセットを開くときに指定するオープン・モードでのtfcd.hヘッダー・ファイルには、以下のようなマクロが定義されています。

```
#define TFCD_OPEN_INPUT      0    /* OPEN INPUT */
#define TFCD_OPEN_OUTPUT    1    /* OPEN OUTPUT */
#define TFCD_OPEN_INOUT     2    /* OPEN I-O */
#define TFCD_OPEN_EXTEND    3    /* OPEN EXTEND */
```

データセットを正常に開いた場合、tfcd_get_status()で確認できる状態コードに00が保存されます。ただし、すでに開いているデータセットを再度開こうとすると、エラーが発生します。

- プロトタイプ

```
int tfcd_open(void *fcd, int open_mode, int flags);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
open_mode(入力)	データセットのオープン・モードです。 – INPUT – OUTPUT – INOUT – EXTEND
flags(入力)	データセットを開くときに指定できる属性です。 – NO REWIND

- 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、データセットを正常にオープンしたのかを確認します。

D.2.2. tfcd_close()

FCD構造体に記述されているデータセットを閉じます。データセットのクローズは、アプリケーションでデータセットの読み書きなどのI/Oを行った後、アプリケーションと物理的なデータセットとの接続を切断する作業です。

データセットを正常に閉じた後は、tfcd_get_status()で確認できる状態コードに00が保存されます。閉じられたデータセットは、tfcd_open() APIを使用して再度開くことができます。

- プロトタイプ

```
int tfcd_close(void *fcd, int flags);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
flags(入力)	データセットを閉じるときに指定できる属性です。 – WITH LOCK

- 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、データセットを正常にクローズしたのかを確認します。

D.3. レコード・アクセス

D.3.1. tfcd_start()

相対データセット、または索引データセットで次に読み取るレコードの位置に移動させます。移動させるレコードの位置は、相対データセットの場合、tfcd_set_relative_key() APIを呼び出して相対レコード番号を指定します。一方、索引データセットの場合は、レコード・ポインター内にあるキーの位置に予めキー値を指定しておきます。

START APIの実行時に、キー値と同じレコードを検索するか、あるいはキー値より大きいか等しいレコードを検索するかを指定することができます。tfcd.hヘッダー・ファイルに以下のようなマクロが定義されています。

```
#define TFCD_START_DEFAULT      0    /* START */
#define TFCD_START_EQUAL       1    /* START (equal to full length) */
#define TFCD_START_GTEQ       2    /* START (not less than) */
```

VSAMデータセットの場合は、該当するデータセットのデフォルトのレコード・キー以外に副レコード・キーを追加で定義できます。副レコード・キーを使用してStart APIを実行するときは、tfcd_set_key_id() APIを呼び出して、次回Start APIで使用するキーIDを指定します。キーIDは、メインCOBOLアプリケーション・プログラムのSELECT文で副レコード・キーを指定した順に番号が与えられます。

● プロトタイプ

```
int tfcd_start(void *fcd, char *record, int flags);
```

● パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
record(入出力)	移動させるレコードの位置が保存されているレコード・バッファ・ポインターです。
flags(入力)	データセットを開始するときに指定できる属性です。 – EQUAL – GTEQ

● 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、データセットを正常にオープンしたのかを確認します。

D.3.2. tfcd_read()

データセットのレコードを順次に読み取るか、キーを指定してランダムに読み取ります。ランダムにレコードを読み取る場合、相対データセットでは、tfcd_set_relative_key() APIを呼び出して相対レコード番号を指定します。一方、索引データセットでは、レコード・ポインター内にあるキーの位置に予めキー値を指定しておきます。

VSAMデータセットの場合は、該当するデータセットのデフォルトのレコード・キー以外に副レコード・キーを追加で定義できます。副レコード・キーを使用してランダムにレコードを読み取る場合は、tfcd_set_key_id() APIを呼び出して次回READ APIで使用するキーIDを指定します。キーIDは、メインCOBOLアプリケーション・プログラムのSELECT文で副レコード・キーを指定した順に番号が与えられます。

tfcd.hヘッダー・ファイルには、Read APIの実行時にレコードを順次に読み取るか、ランダムに読み取るかを指定するマクロが以下のように定義されています。

```
#define TFCD_READ_DEFAULT      0    /* READ */
#define TFCD_READ_NEXT        1    /* READ (sequential) */
```

可変長レコード・データセットの場合、Read APIを使用してデータセットのレコードを読み取ると、tfcd_get_rec_len() APIを呼び出して読み取ったレコード長を取得できます。取得したレコード長は、RDWを除いたレコードのデータ部分の長さです。

- プロトタイプ

```
int tfcd_read(void *fcd, char *record, int flags);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体のアドレスです。
record(入出力)	データセットから読み取ったレコードが保存されるレコード・バッファ・ポインターです。
flags(入力)	データセットを読み取る時に指定できる属性です。 – DEFAULT – NEXT

- 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、データセットを正常に読み取ったのかを確認します。

D.3.3. tfcd_write()

データセットに新しいレコードを書き込むか、または既存のレコードを再書き込みします。

順次データセットの場合はデータセットの最後にレコードを書き込み、相対データセットの場合はtfcd_set_relative_key() APIを呼び出して保存される相対レコード番号を指定します。索引データセットの場合は、レコード・ポインター内にあるキー順にレコードが保存されます。

tfcd.hヘッダー・ファイルには、Write APIの実行時に新しいレコードを書き込むか、既存のレコード内容を更新するかを指定するマクロが以下のように定義されています。

```
#define TFCD_WRITE_DEFAULT      0    /* WRITE */
#define TFCD_WRITE_REWRITE      1    /* REWRITE */
```

可変長レコード・データセットの場合、Write APIを使用してデータセットのレコードを書き込む前に、レコード長をtfcd_set_rec_len() APIを呼び出して指定することができます。そのとき、レコード長はRDWを除いたレコードのデータ部分の長さを指定します。

- プロトタイプ

```
int tfcd_write(void *fcd, char *record, int flags);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
record(入力)	データセットに書き込むレコードが保存されているレコード・バッファ・ポインターです。
flags(入力)	データセットを書き込むときに指定できる属性です。 – WRITE – REWRITE

- 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、データセットを正常に書き込んだのかを確認します。

D.3.4. tfcd_writea()

レコードの先頭にASA制御文字を指定し、データセットに新しいレコードを書き込みます。tfcd.h ヘッダー・ファイルには、Writea APIの実行時にASA制御文字を指定するマクロが以下のように定義されています。

```
#define TFCD_WRITEA_AFTER      0    /* AFTER */
#define TFCD_WRITEA_BEFORE    1    /* BEFORE */

#define TFCD_WRITEA_TAB       2    /* TAB */
#define TFCD_WRITEA_PAGE      4    /* PAGE */
#define TFCD_WRITEA_MNEMONIC  8    /* MNEMONIC */
```

- プロトタイプ

```
int tfcd_writea(void *fcd, char *record, int lcount, int flags);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
record(入力)	データセットに書き込むレコードが保存されているレコード・バッファ・ポインターです。

パラメータ	説明
lcount(入力)	レコードを出力する前に移動させる行数です。
flags(入力)	データセットを書き込むときに指定できる属性です。

- 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、データセットを正常に書き込んだのかを確認します。

D.3.5. tfcd_delete()

相対データセットや索引データセットの場合、直前に読み取ったレコードを削除します。

- プロトタイプ

```
int tfcd_delete(void *fcd, char *record, int flags);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
record(入力)	データセットから削除するレコードが保存されているレコード・バッファ・ポインターです。
flags(入力)	データセットを削除するときに指定できる属性です。

- 戻り値

正常に関数が実行された場合、0が返されます。状態コード値を確認して、レコードを正常に削除したのかを確認します。

D.4. 特殊な操作

D.4.1. tfcd_is_file_open()

FCD構造体に記述されているデータセットが開かれているかを確認します。メインCOBOLアプリケーション・プログラムで開いたデータセットのDD名を変更する前に、データセットが開かれているかどうかを確認するために使用します。

- プロトタイプ

```
int tfcd_is_file_open(void *fcd);
```


- パラメータ

パラメータ	説明
fcd(入力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。

- 戻り値

データセットが開かれていれば1、そうでない場合は0が返されます。

D.4.2. tfcd_get_status()

データセットI/Oを実行した後、エラーの発生有無を確認するためにデータセットの状態コードを取得します。一般的に状態コード値がスペースであると、データセットへのI/O機能が正常に実行されたこととなります。

データセットI/Oの実行時に深刻なシステム・エラーやOpenFrameライブラリ・エラーが発生すると、エラーメッセージと同時にプログラムが終了されます。ただし、End of File(ファイルの終了)などの一般的なデータセットI/Oエラーの場合は、プログラムが終了せずに適切な状態コード値が設定された後、I/O関数が正常に返されます。したがって、I/O関数を実行した後は、必ずtfcd_get_status()関数を使用して、エラーの発生有無を確認する必要があります。

データセットI/O関数で設定できるデータセットの状態コード値は、MicroFocus Server Expressのオンライン・マニュアル(<http://supportline.microfocus.com/Documentation/books/sx40sp2/sx40indx.htm>)を参照してください。

- プロトタイプ

```
int tfcd_get_status(void *fcd, char *status);
```

- パラメータ

パラメータ	説明
fcd(入力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
status(出力)	データセットの状態情報を保存するバッファです。

- 戻り値

正常に関数が実行された場合、0が返されます。

D.4.3. tfcd_get_file_name()

FCD構造体に記述されているデータセットのDD名を取得します。開かれているデータセットと閉じられているデータセットの両方に使用できるAPIです。

- プロトタイプ

```
int tfcd_get_file_name(void *fcd, char *file_name);
```

- パラメータ

パラメータ	説明
fcd(入力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
file_name(出力)	データセットのDD名を保存するバッファです。

- 戻り値

正常に関数が実行された場合、0が返されます。

D.4.4. tfcd_get_rec_len()

FCD構造体に記述されているデータセットのレコード長を取得します。固定長データセットが最初にかかれた時点で全体データセットの固定レコード長を取得する場合や、可変長データセット・レコードをREAD APIで読み取った後、読み取ったレコードの長さを取得する場合に使用できます。

可変長レコード・データセットのレコード長は、RDWを除いたレコードのデータ部分の長さです。

- プロトタイプ

```
int tfcd_get_rec_len(void *fcd, int *reclen);
```

- パラメータ

パラメータ	説明
fcd(入力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
reclen(出力)	データセットのレコード長を保存するバッファです。

- 戻り値

正常に関数が実行された場合、0が返されます。

D.4.5. tfcd_set_file_name()

FCD構造体に記述されているデータセットのDD名を指定します。メインCOBOLアプリケーション・プログラムで開いたデータセットのDD名を変更し、新しいデータセットを開くために使用します。同APIを使用する前に、開かれていたデータセットはClose APIを使用して閉じる必要があります。

- プロトタイプ

```
int tfcd_set_file_name(void *fcd, char *file_name);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
file_name(入力)	データセットに指定するDD名です。

- 戻り値

正常に関数が実行された場合、0が返されます。

D.4.6. tfcd_set_relative_key()

FCD構造体に記述されているデータセットの相対キーを指定します(RELATIVE)。相対データセットの場合、tfcd_set_relative_key()で指定した相対キーの位置に新しいレコードを保存するか、保存されていた既存のレコードを読み取ります。

- プロトタイプ

```
int tfcd_set_relative_key(void *fcd, int relative_key);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
relative_key(入力)	相対データセットの相対キーです。

- 戻り値

正常に関数が実行された場合、0が返されます。

D.4.7. tfcd_set_key_id()

FCD構造体に記述されているデータセットのキーIDを指定します(INDEXED)。

副レコード・キーが追加で定義されているデータセットに対し、副レコード・キーを使用してランダムにレコードを読み取る場合、tfcd_set_key_id() APIを呼び出して次回Read APIで使用するキーIDを指定します。

- プロトタイプ

```
int tfcd_set_key_id(void *fcd, int key_id);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
key_id(入力)	データセットに指定するキーIDです。

- 戻り値

正常に関数が実行された場合、0が返されます。

D.4.8. tfcd_set_rec_len()

FCD構造体に記述されているデータセットのレコード長を指定します。可変長データセット・レコードをWrite APIを使用して書き込む前に、レコード長を指定する場合に使用します。可変長レコード・データセットのレコード長は、RDWを除いたレコードのデータ部分の長さを指定します。

- プロトタイプ

```
int tfcd_set_rec_len(void *fcd, int reclen);
```

- パラメータ

パラメータ	説明
fcd(入出力)	メインCOBOLアプリケーション・プログラムから渡されたFCD構造体アドレスです。
reclen(入力)	データセットに指定するレコード長です。

- 戻り値

正常に関数が実行された場合、0が返されます。

付録 E. OpenFrame VSAMのパフォーマンス最適化

本付録では、OpenFrame VSAMのパフォーマンスの最適化方法について説明します。

E.1. 概要

OpenFrame 7 VSAM(TSAM)は、JCLユーティリティのIDCAMSまたはコマンド・ライン・ツールのidcamsを使用して定義することができます。

OpenFrame 7 VSAMはRDBテーブルとして定義されます。テーブルの構造は、ds.TSAM_RDB.COPYBOOK_PATHとして設定されたディレクトリで生成するデータセットと同じ名前のコピーブック・ファイルを参照して定義されます。そのコピーブックは、テーブル定義およびOpenFrameエンジンの動作でのみ使用されるため、移行スキーマやユーザー・プログラムのFD定義と同じである必要はありません。

以下は、フィールド・タイプに応じて定義される列タイプです。

フィールド・タイプ	列タイプ	備考
文字(X、N)	VARCHAR	
Zoned/Packed decimal, binary(COMP,COMP-4,COMP-5)	NUMBER	
Float(COMP-1)	BINARY_FLOAT	
Double(COMP-2)	BINARY_DOUBLE	
EBCDIC(EBC)	NVARCHAR2	X、Aタイプでのみ使用できます。 EBCDICソートを使用するには、EBC タイプを使用する必要があります。 例) PIC X(10) EBC.

OpenFrame 7 VSAMは、ユーザーの必要に応じてデータが適切に列化されてテーブルに格納されるため、RDBの利点を最大限に活用し、他の現代的なシステムおよびアプリケーションと簡単に連携できるというメリットがあります。

ただし、COBOLプログラムの入出力では、列化されたデータを読み取る時はバイト配列でシリアル化し、書き込むときは逆シリアル化するプロセスが必要となるため、追加コストが発生します。したがって、実行時間が重要なバッチ業務の場合は、パフォーマンスの最適化が必要になる場合があります。

ds.TSAM_RDB.PRINT_DEBUG_MSG_ONの設定をYESにすると、OpenFrame 7 VSAMは、プログラム命令に従ってエンジンから要求されたSQLや実行時間を出力します。

以下のログの例です。

```
[START]
* SQL
  SELECT /*+ INDEX_ASC (BASE "PROD_VSAM_KEY") */ SM_KEY, STYLE_NUMBER, YEAR, SIZE,
  SIZE_SCALE_PCT, EACH_PRICE_1, EACH_PRICE_2, EACH_PRICE_3, EACH_PRICE_4 FROM
"PROD_VSAM" BASE WHERE ("SM_KEY", "STYLE_NUMBER", "YEAR", "SIZE") >= (:SM_KEY,
:STYLE_NUMBER, :YEAR, :SIZE) AND ROWNUM = 1 ORDER BY "SM_KEY", "STYLE_NUMBER",
"YEAR", "SIZE" ASC
* KEY
  SM_KEY(50), STYLE_NUMBER(000100009), YEAR(22), SIZE()
* Current Time   : 2023-03-06 11:26:03
* Execution Time : 0.0020s

[OPEN]
* SQL
  SELECT /*+ INDEX_ASC (BASE "PROD_VSAM_KEY") */ SM_KEY, STYLE_NUMBER, YEAR, SIZE,
  SIZE_SCALE_PCT, EACH_PRICE_1, EACH_PRICE_2, EACH_PRICE_3, EACH_PRICE_4 FROM
"PROD_VSAM" BASE WHERE ("SM_KEY", "STYLE_NUMBER", "YEAR", "SIZE") >= (:SM_KEY,
:STYLE_NUMBER, :YEAR, :SIZE) ORDER BY "SM_KEY", "STYLE_NUMBER", "YEAR", "SIZE"
ASC
* KEY
  SM_KEY(50), STYLE_NUMBER(000100009), YEAR(22), SIZE(1X          )
* Current Time   : 2023-03-06 11:26:03
* Execution Time : 0.0441s
```

上記の例で、実行時間が長いSQLを対象に実行計画などを確認してチューニングポイントを見つけます。

E.2. その他のパフォーマンス最適化方法

以下は、その他のパフォーマンス最適化方法です。

- **コピーブック・フィールドの数を減す**

列の数が多くなるほどSQLの処理性能が低下し、OpenFrameエンジンにも負荷がかかります。したがって、フィールドを結合して列の数を減らすことで、入出力の速度を向上させることができます。

- **コピーブック・フィールドをX型として作成**

ゾーン10進数、パック10進数、整数型のフィールドの場合、COBOL規約に適合するデータ形式とRDBに格納される数値形式が異なるため、入出力時にデータ変換が必要となります。したがって、このようなフィールドが多くなると負荷が大きくなります。

一方、X型のフィールドはデータ変換なしで、VARCHAR列にそのまま格納されるため、比較的高速な入出力が可能です。

上記の2つのケースを組み合わせ、コピーブックをインデックスキーとして使用するX型のフィールド1つと、データの残りの部分に対するX型のフィールド1つに単純化すれば、COBOLプログラムの入出力パフォーマンスを向上させることができます。ただし、その分だけSQLを使用したデータ操作が難しくなるデメリットもあります。

● PAGESIZEオプションの使用

一部のユーザー・プログラムでは、レコードの総数が多いが、READ命令で一度に読み取られるレコード数が少ないため、READ命令が繰り返されることがあります。OpenFrameエンジンは、READコマンドが繰り返される場合を正常と判断し、READ時にキー以降のレコードをすべて取得するクエリを実行しますが、このクエリはレコード数が多いほど実行速度が遅くなるため、動作効率が低下します。

JCLのDCBに以下の例のようにPAGESIZE=nオプションを与えると、一度に読み取るレコード数を指定できるため、ジョブに応じてこのオプションを適切に指定して実行時間を短縮することが可能です。

```
//JOB0      EXEC PGM=READTEST
//DDONE     DD DSN=TEST.VSAM,DISP=OLD,
//          DCB=(PAGESIZE=100)
//SYSOUT    DD SYSOUT=*
//SYSPRINT  DD SYSOUT=*
```


索引

A

AIX, 33
Alias Entry, 44
Allocation Group, 45
Alternate Index, 44
AMS(Access Method Service), 7
amsu_assoc(), 57
amsu_candidate_catalog(), 54
amsu_default_volume(), 51
amsu_delete(), 56
amsu_filename2(), 56
amsu_filepath2(), 55
amsu_finalize(), 50
amsu_info(), 57
amsu_initialize(), 50
amsu_master_catalog(), 54
amsu_search_entries(), 52
amsu_use_catalogs(), 52
amsu_volume_list(), 51
amsu_volume_path(), 52
Association Group, 45
Attribute Group, 45

B

BDAM(Basic Direct Access Method), 2
BLKSIZE, 16
BPAM(Basic Partitioned Access Method), 2
BPAMアクセス方式
 BLDL, 21
 FIND, 21
 STOW, 21
BSAM(Basic Sequential Access Method), 2

D

Data Component, 44
DCBハンドリングAPI

tdcb_discard_handle(), 75
tdcb_get_handle(), 74
tdcb_get_type(), 76
tdcb_maxlrecl(), 77
tdcb_nvsm_info(), 77
tdcb_print_info(), 75

DCB処理API

tdcb_allocate(), 87
tdcb_concatenate(), 88
tdcb_get_blksize(), 82
tdcb_get_dsname(), 79
tdcb_get_flags(), 80
tdcb_get_key_info(), 82
tdcb_get_volser(), 80
tdcb_get_vsamtype(), 83
tdcb_set_blksize(), 85
tdcb_set_keylen(), 86
tdcb_set_lrecl(), 84
tdcb_set_recfm(), 85
tdcb_unallocate(), 89

DCB構造体, 8, 15

dsdeleteツール, 26

DSORG, 16

E

ESDS, 3, 29

F

FCDハンドリングAPI

tfcd_close(), 94
tfcd_delete(), 98
tfcd_get_file_name(), 99
tfcd_get_rec_len(), 100
tfcd_get_status(), 99
tfcd_is_file_open(), 98
tfcd_open(), 93
tfcd_read(), 95
tfcd_set_file_name(), 100
tfcd_set_key_id(), 101
tfcd_set_rec_len(), 102
tfcd_set_relative_key(), 101
tfcd_start(), 94

tfcd_write(), 96
tfcd_writea(), 97

G

GDG, 43
GDG Group, 45
GDS, 43

H

History Group, 44

I

Index Component, 44
ISAM(Indexed Sequential Access Method), 2

J

JCL(Job Control Language), 7

K

KSDS, 3, 29

L

LDS, 3, 30
LRECL, 16

N

Non-VSAM Data Set, 43
Non-VSAM Group, 45

P

Path Entry, 44
PDSメンバーの処理, 21
Protection Group, 44

Q

QSAM(Queued Sequential Access Method), 2

R

RECFM, 16
RRDS, 3, 29

S

SMS Group, 44

T

tcfh_bldl(), 67
tcfh_bldl_info(), 68
tcfh_close(), 62
tcfh_delete(), 67
tcfh_find_by_bldl(), 69
tcfh_find_by_name(), 70
tcfh_open(), 60
tcfh_read(), 64
tcfh_rewrite(), 66
tcfh_start(), 63
tcfh_stow(), 70
tcfh_write(), 65
tdcb_allocate(), 87
tdcb_concatenate(), 88
tdcb_discard_handle(), 75
tdcb_get_blksize(), 82
tdcb_get_dsname(), 79
tdcb_get_flags(), 80
tdcb_get_handle(), 74
tdcb_get_key_info(), 82
tdcb_get_type(), 76
tdcb_get_volser(), 80
tdcb_get_vsamtype(), 83
tdcb_maxlrecl(), 77
tdcb_nvsm_info(), 77
tdcb_print_info(), 75
tdcb_set_blksize(), 85
tdcb_set_keylen(), 86
tdcb_set_lrecl(), 84
tdcb_set_recfm(), 85
tdcb_unallocate(), 89
tfcd_close(), 94
tfcd_delete(), 98
tfcd_get_file_name(), 99
tfcd_get_rec_len(), 100
tfcd_get_status(), 99
tfcd_is_file_open(), 98
tfcd_open(), 93

tfcd_read(), 95
tfcd_set_file_name(), 100
tfcd_set_key_id(), 101
tfcd_set_rec_len(), 102
tfcd_set_relative_key(), 101
tfcd_start(), 94
tfcd_write(), 96
tfcd_writea(), 97

U

User Catalog, 44

V

Volumes Group, 45
VSAM Cluster, 44
VSAM(Virtual Storage Access Method), 3
VSAMデータセット, 27
 ESDS, 30
 KSDS, 30
 RRDS, 31
 VRDS, 31
VSAMデータセットのパラメータ
 CATALOG, 35
 CONTROLINTERVALSIZE, 36
 CYLINDERS, 35
 INDEXED, 35
 KEYS, 35
 KILOBYTES, 35
 MEGABYTES, 35
 NOINDEXED, 35
 NUMBERED, 35
 RECORDS, 35
 RECORDSIZE, 35
 SPANNED, 36
 TRACKS, 35
 VOLUMES, 35
VSAMデータセットの定義, 35
VSAMデータ形式, 27

あ

アクセス・パスの定義, 38
アクセス・モード, 9

エイリアス項目, 42
オープン・モード, 8

か

解除(Unallocate), 8
拡張モード(Extend Mode), 9
仮想記憶アクセス方式(VSAM), 3
可変長レコード形式, 14, 31
基本区分アクセス方式(BPAM), 2
基本順次アクセス方式(BSAM), 2
基本直接アクセス方式(BDAM), 2
行順次レコード形式, 15
区分データセット(PDS), 19
固定長レコード形式, 13
カタログ, 41
カタログ・エイリアスの定義, 46
カタログ・エントリのリスト, 38
カタログ・エントリのリスト化, 46
カタログ管理API
 amsu_assoc(), 57
 amsu_candidate_catalog(), 54
 amsu_default_volume(), 51
 amsu_delete(), 56
 amsu_filename2(), 56
 amsu_filepath2(), 55
 amsu_finalize(), 50
 amsu_info(), 57
 amsu_initialize(), 50
 amsu_master_catalog(), 54
 amsu_search_entries(), 52
 amsu_use_catalogs(), 52
 amsu_volume_list(), 51
 amsu_volume_path(), 52
キー順データセット(KSDS), 30
クラスター, 29
クラスターの概念, 35
コンポーネント, 29

さ

索引付き順次アクセス方式(ISAM), 2
出力モード(Output Mode), 9
順次アクセス(Sequential Access), 9, 10

- 順次データセット, 18
- 制御インターバル, 27
 - クラスター, 29
 - コンポーネント, 29
 - スパン・レコード, 28
 - スプリット, 28
 - 制御領域, 28
- 制御領域(Control Area), 28
- 世代データ・グループ(GDG), 22
- 選択制御文字, 15
- 絶対世代番号, 23
- 相対世代番号, 24
- 相対レコード・データセット(RRDS), 31
- システム・カタログ, 43
- ストレージ・クラス, 5
- スパン・レコード(Spanned Records), 28
- スフィア, 34
- スプリット(Splits), 28

た

- 待機順次アクセス方式(QSAM), 2
- 代替索引, 33
- 代替索引の管理, 37
- 代替索引の定義, 37
- 中間コミット, 40
- データセットのコピー, 38
- データセットの削除, 39
- データセットの出力, 39
- 動的アクセス(Dynamic Access), 9
- テープ・ボリュームのロック, 26
- データセットI/O API
 - tcfh_bldl_info(), 68
 - tcfh_bldl(), 67
 - tcfh_close(), 62
 - tcfh_delete(), 67
 - tcfh_find_by_bldl(), 69
 - tcfh_find_by_name(), 70
 - tcfh_open(), 60
 - tcfh_read(), 64
 - tcfh_rewrite(), 66
 - tcfh_start(), 63
 - tcfh_stow(), 70

- tcfh_write(), 65
- データセットのオープン, 8
- データセットのクローズ, 8
- データセットの割り当て
 - AMS, 7
 - JCL, 7
- データセット・アクセス方式
 - BDAM, 2
 - BPAM, 2
 - BSAM, 2
 - ISAM, 2
 - QSAM, 2
 - VSAM, 3
- データ・クラス, 5
- トランザクション, 40
- トランザクション・インターフェース
 - COMMIT, 40
 - ROLLBACK, 40

な

- 入出力モード(Input/Output Mode), 9
- 入力順データセット(ESDS), 30
- 入力モード(Input Mode), 8

は

- 非VSAMデータセット, 13
- 非VSAMデータセットのスペースの割り当て, 16
- 不定長レコード形式, 15
- ファイル・ステータス・コード, 71
- ブラウジング, 39
- ブラウジング・インターフェース
 - READ_NEXT, 39
 - READ_PREV, 39
 - RESET_BR, 40
 - START_BR, 39

ま

- マスター・カタログ, 42
- マネジメント・クラス, 5

や

- ユーザー・カタログ, 42

ユーザー・カタログの削除, 47

ユーザー・カタログの定義, 45

ら

ランダムアクセス(Random Access), 9, 10

レコード・アクセス, 10

レコード形式, 13

 可変長, 14

 行順次, 15

 固定長, 13

 不定長, 15

わ

割り当て(Allocate), 7

